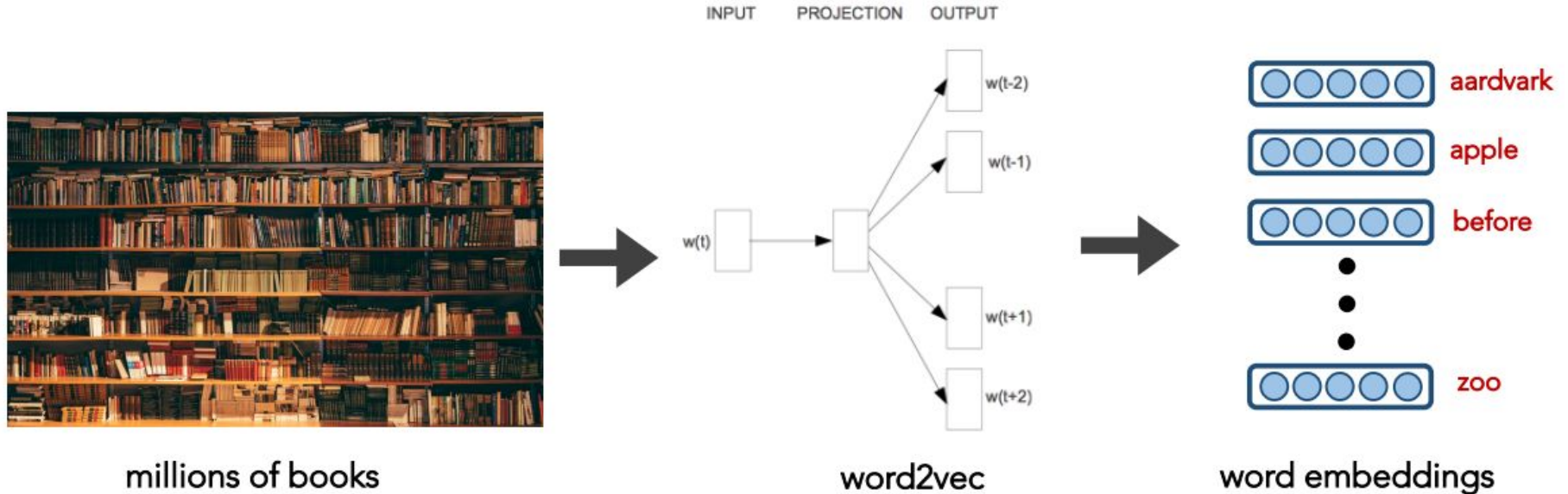
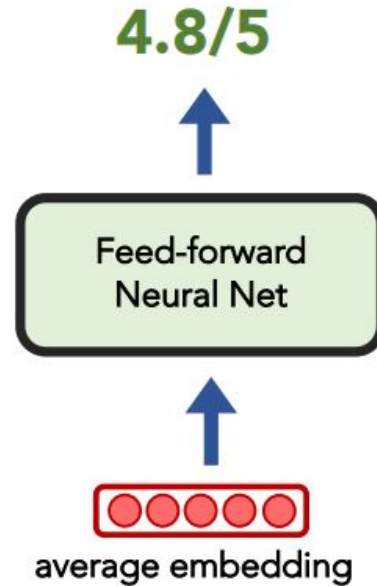
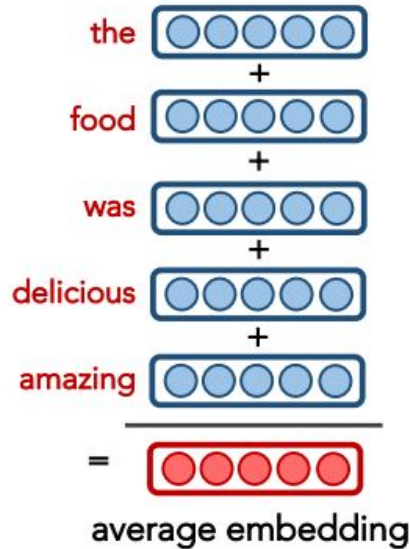


Previously, we learned about **word embeddings**



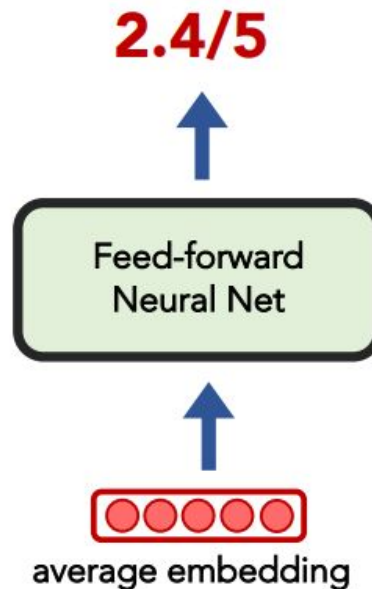
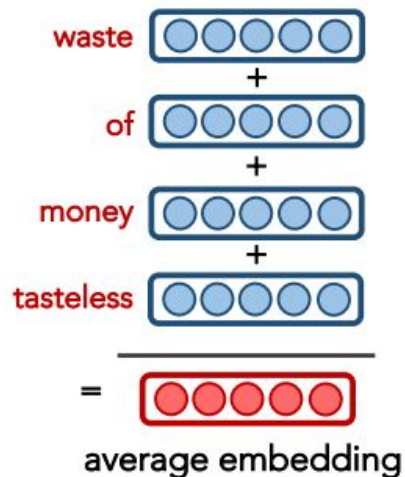
Word Embeddings - NN Based Sentiment Analysis

"The food was delicious. Amazing!" → 4.8/5 * yelp

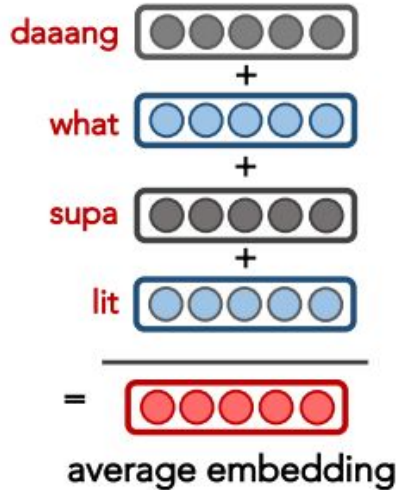


Word Embeddings - NN Based Sentiment Analysis

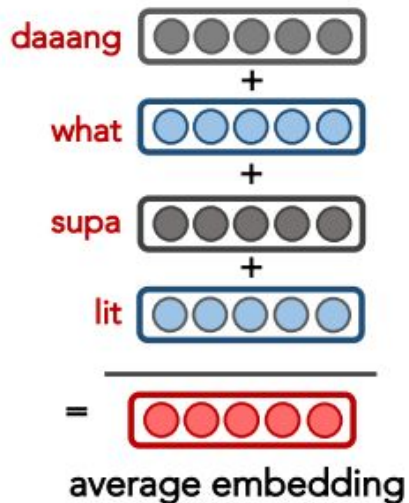
"Waste of money. Tasteless!" → 2.4/5 * yelp



"Daaang. What?! Supa Lit" → 4.9/5  yelp

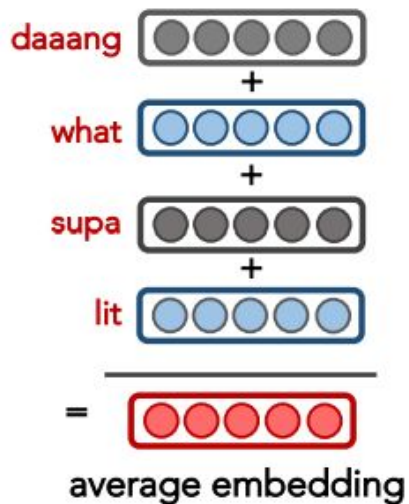


Strengths and weaknesses of word embeddings (type-based)?



Strengths:

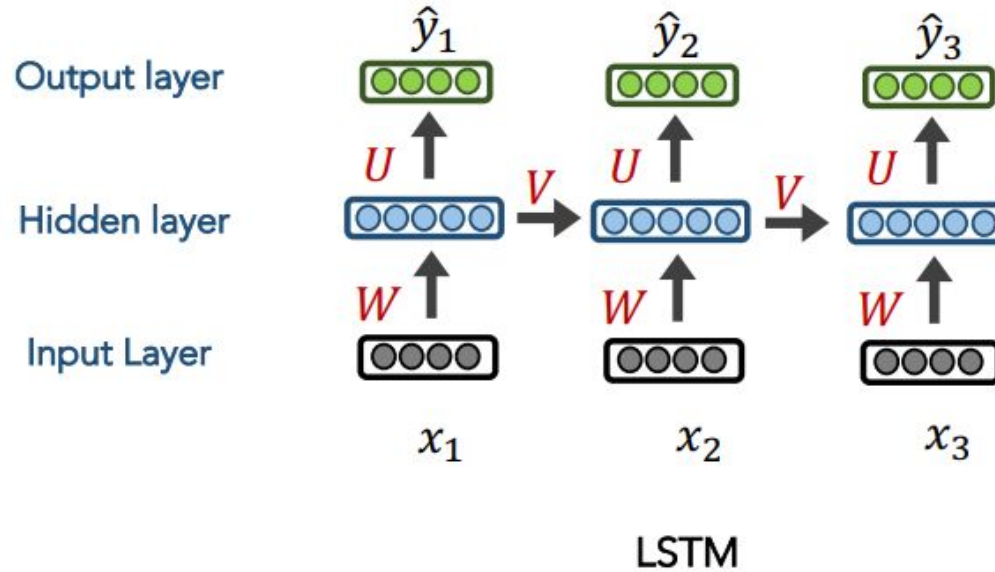
- Leverages tons of existing data
- Don't need to depend on our data to create embeddings



Issues:

- Out-of-vocabulary (OOV) words
- Not tailored to this dataset

Word Embeddings - RR Based Sentiment Analysis



Word Embeddings - RR Based Sentiment Analysis

1. Sarma

★★★★★ 895

\$\$\$ • Middle Eastern, Tapas/Small Plates, Mediterranean

2. Chalawan

★★★★★ 108

Thai, Indonesian, Singaporean

3. Gustazo Cuban Kitchen & Bar

★★★★★ 162

Cuban, Tapas/Small Plates, Bars

4. Posto

★★★★★ 912

\$\$ • Italian, Pizza

5. Avenue Kitchen + Bar

★★★★★ 81

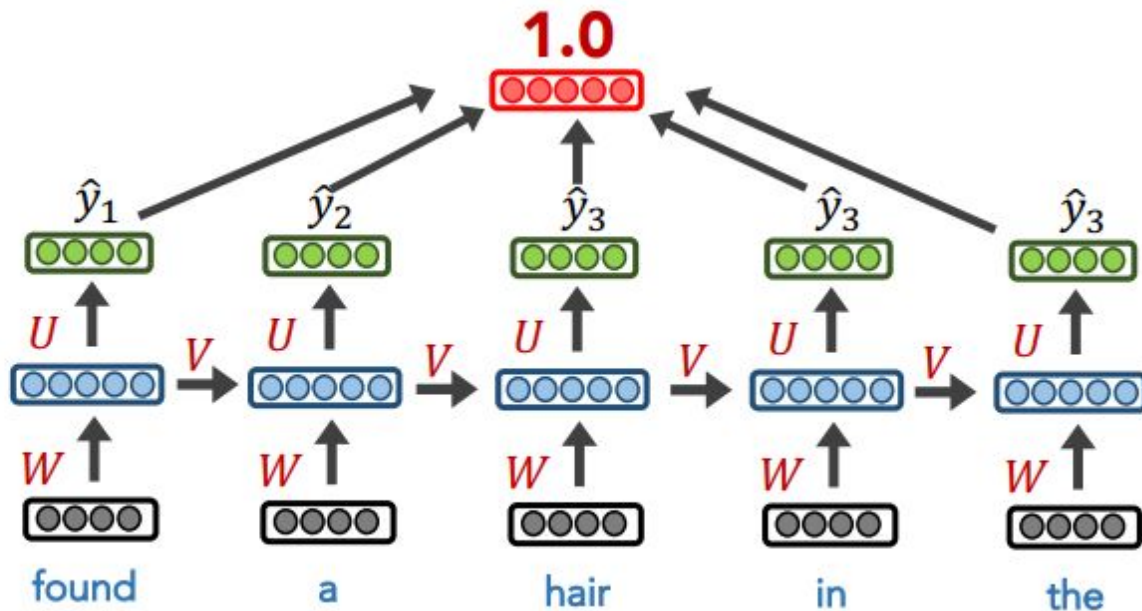
\$\$ • Beer Bar, Pizza, Cocktail Bars

•
•
•

7192. Underdog Hot Chicken

★★★★★ 45

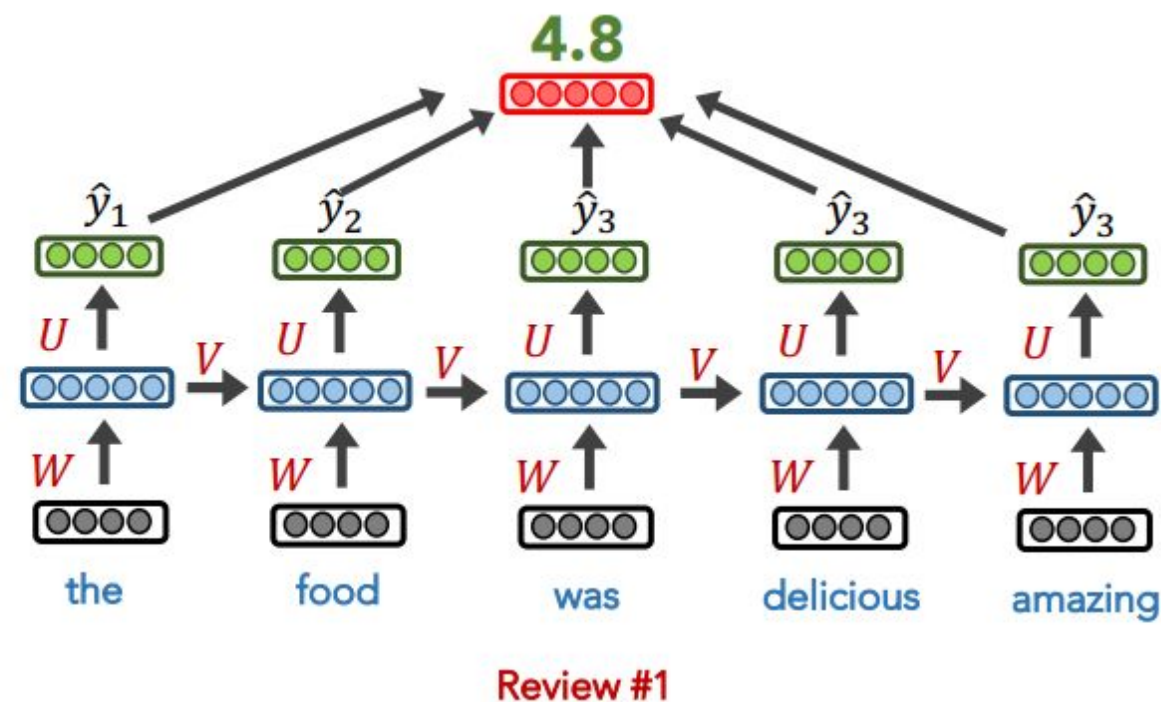
Chicken Wings, Chicken Shop



Review #53,781

Word Embeddings - RNN Based Sentiment Analysis

- 1. Sarma**
★★★★ 895
\$\$\$ • Middle Eastern, Tapas/Small Plates, Mediterranean
- 2. Chalawan**
★★★★ 108
Thai, Indonesian, Singaporean
- 3. Gustazo Cuban Kitchen & Bar**
★★★★ 162
Cuban, Tapas/Small Plates, Bars
- 4. Posto**
★★★★ 912
\$\$ • Italian, Pizza
- 5. Avenue Kitchen + Bar**
★★★★ 81
\$\$ • Beer Bar, Pizza, Cocktail Bars
- ...
- 7192. Underdog Hot Chicken**
★★★★ 45
Chicken Wings, Chicken Shop



RNN Based Contextual Embedding

1. Sarma

★★★★★ 895

\$\$\$ • Middle Eastern, Tapas/Small Plates, Mediterranean

2. Chalawan

★★★★★ 108

Thai, Indonesian, Singaporean

3. Gustazo Cuban Kitchen & Bar

★★★★★ 162

Cuban, Tapas/Small Plates, Bars

4. Posto

★★★★★ 912

\$\$ • Italian, Pizza

5. Avenue Kitchen + Bar

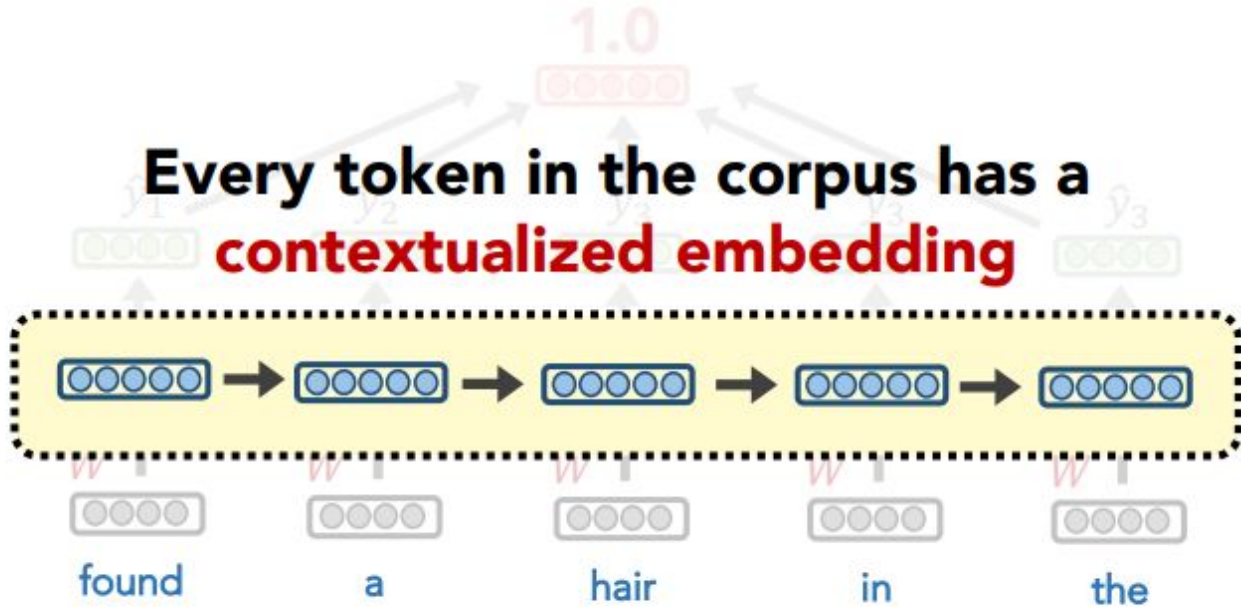
★★★★★ 81

\$\$ • Beer Bar, Pizza, Cocktail Bars

7192. Underdog Hot Chicken

★★★★★ 45

Chicken Wings, Chicken Shop



Review #53,781

RNN Based Contextual Embedding

1. Sarma

★★★★★ 895

\$\$\$ • Middle Eastern, Tapas/Small Plates,
Mediterranean

2. Chalawan

★★★★★ 108

Thai, Indonesian, Singaporean

3. Gustazo Cuban Kitchen & Bar

★★★★★ 162

Cuban, Tapas/Small Plates, Bars

4. Posto

★★★★★ 912

\$ - Italian, Pizza

5. Avenue Kitchen + Bar

★★★★★ 81

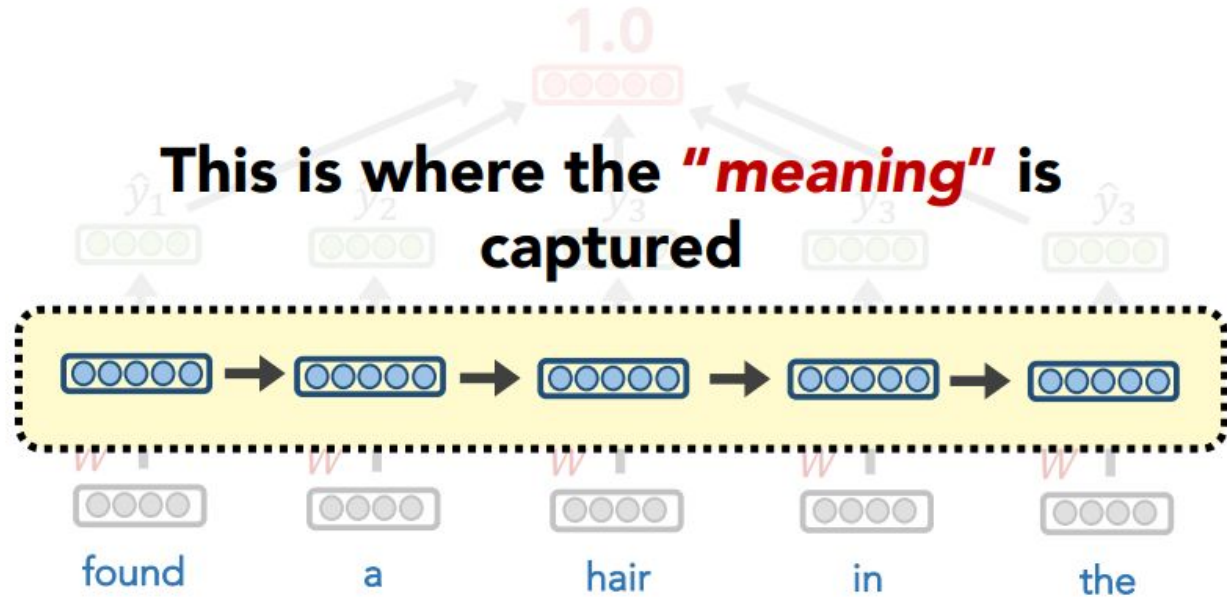
\$ - Beer Bar, Pizza, Cocktail Bars



7192. Underdog Hot Chicken

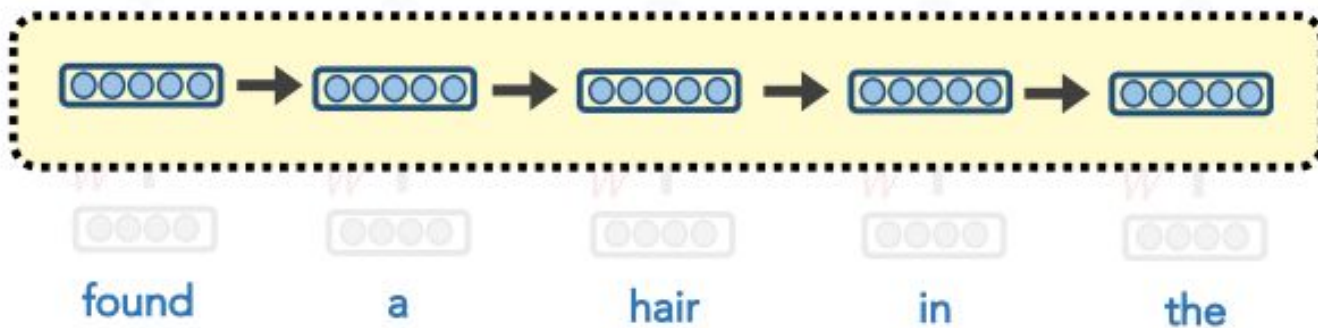
★★★★★ 45

Chicken Wings, Chicken Shop



Review #53,781

Strengths and weaknesses of
contextualized embeddings
(aka token-based)?



Review #53,781

RNN Based Contextual Embedding

Strengths:

- Tailored to your particular corpus
- No out-of-vocabulary (OOV) words



Weaknesses:

- May not have enough data to produce good results
- Have to train new model for each use case
- Can't leverage a wealth of existed text data (millions of books)???

Weaknesses:

- May not have enough data to produce good results
- Have to train new model for each use case
- ~~Can't leverage a wealth of existed text data (millions of books)???~~

WRONG! We can leverage millions of books!

found

a

hair

in

the

Review #53,781

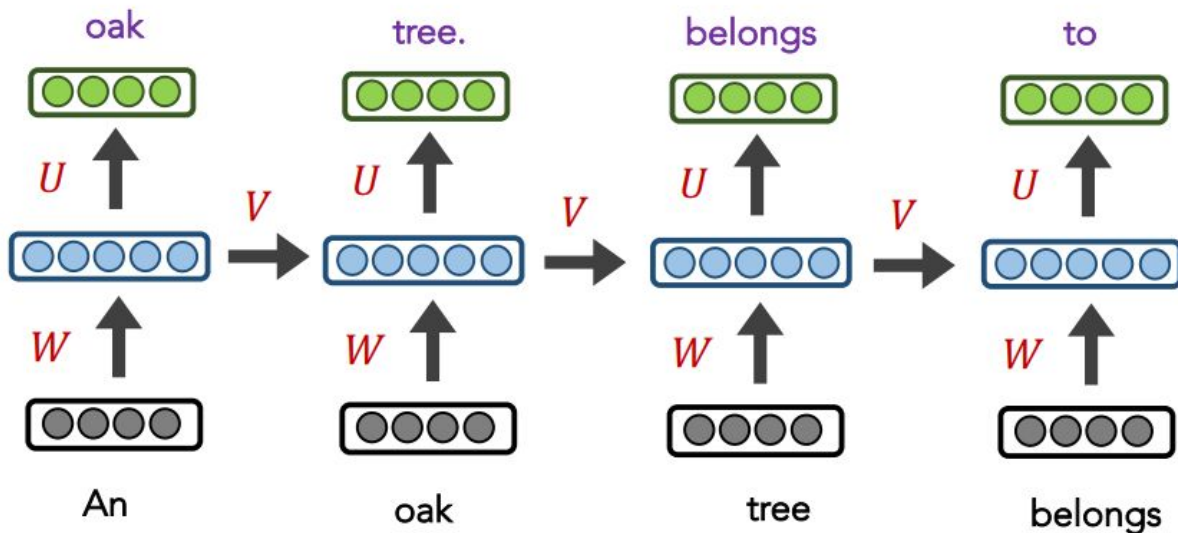
RNN Based Contextual Embedding



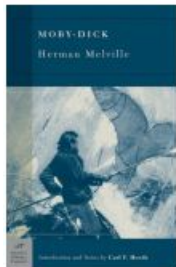
WIKIPEDIA
The Free Encyclopedia

Language Modelling

(let's input 1 million documents)

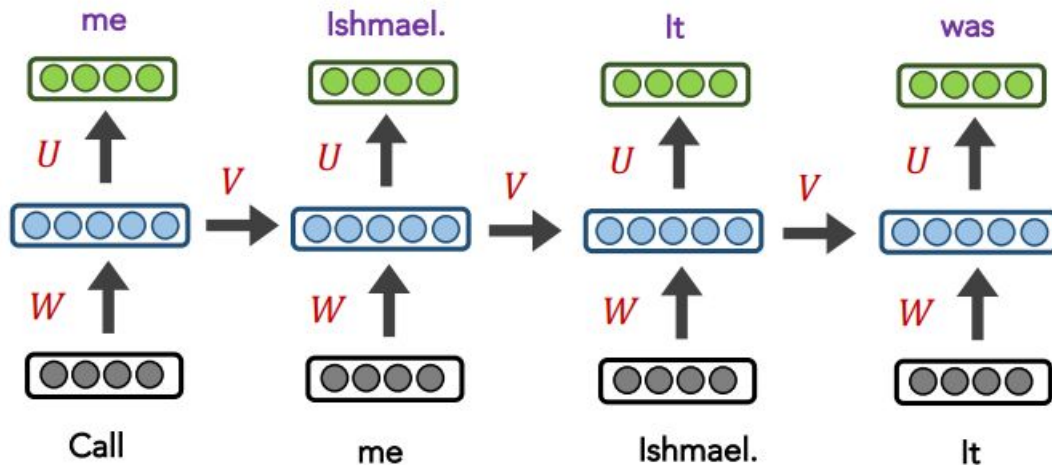


RNN Based Contextual Embedding



Language Modelling

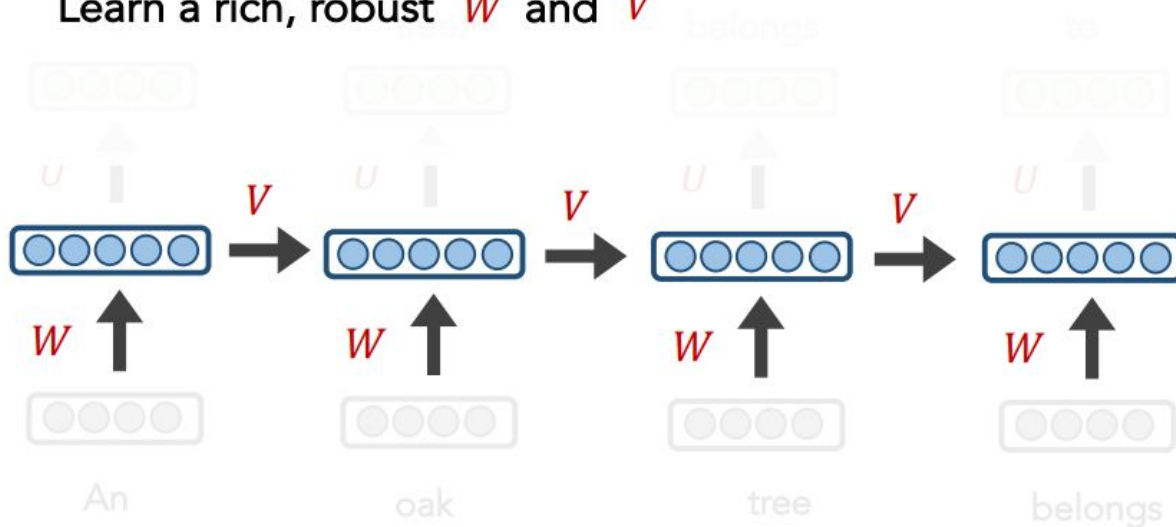
(let's input 1 million documents)



RNN Based Contextual Embedding

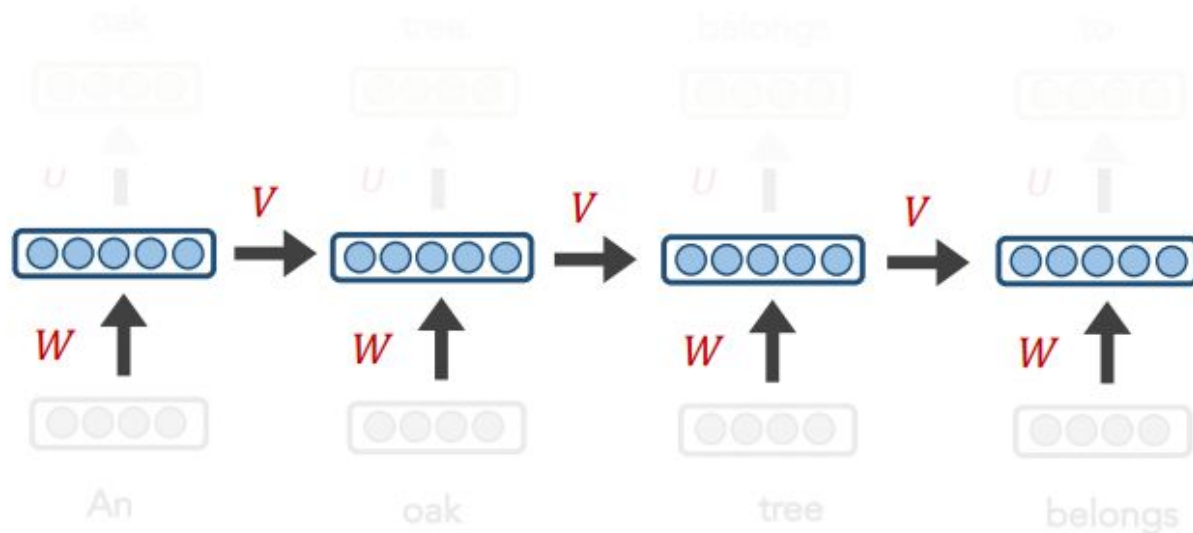
The **contextualized embeddings** for 1 million docs aren't useful to us for a new task (e.g., predicting Yelp reviews), but the learned weights could be!

Learn a rich, robust W and V



RNN Based Contextual Embedding

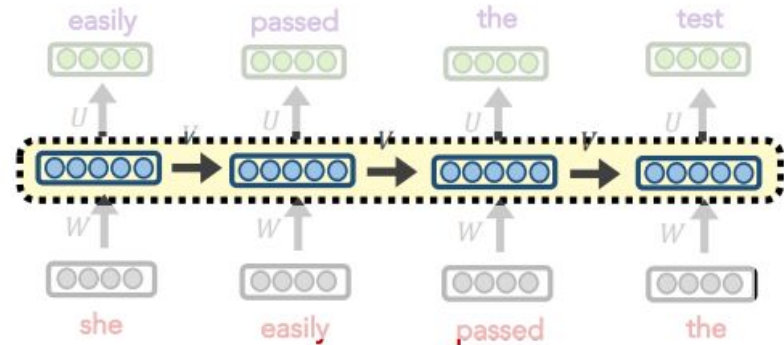
Using these “pre-trained” W and V , we can possibly increase our performance on other tasks (e.g., Yelp reviews), since they’re very experienced with producing/capturing “meaning”



- These models are trained on a **specific task** (e.g., LM, text classification, etc)
- The hidden layer(s) contains the “meaning” and are **very useful**
- We can extract those embeddings if we wish, or grab the learned weights and re-use for another task
- Dominating NLP from **2015 - present**

TOKEN-BASED

contextualized embeddings are distinct for every occurrence of a word, completely dependent on its context

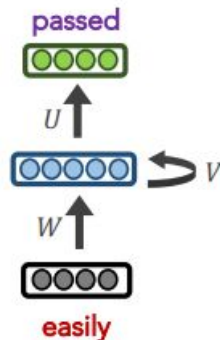


Bidirectional LSTM

- LSTMs are amazing but ultimately only look at **1 word at a time, sequentially**
- Sure, they maintain long-term memory, but **they are short-sighted** in terms of knowing what to hold onto and how to weight each input

TOKEN-BASED

contextualized embeddings are distinct for every occurrence of a word, completely dependent on its context

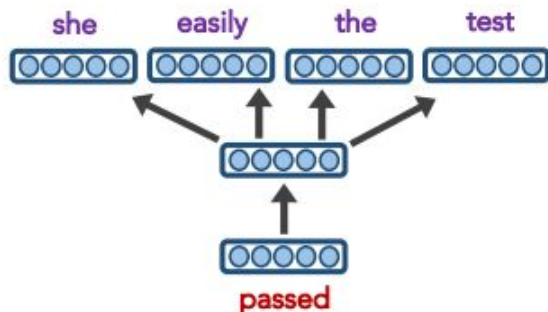


Bidirectional LSTM

Word Embeddings types In General

TYPE-BASED

a single, **global word embedding** for each word, independent of its context.



word2vec (skip-gram)

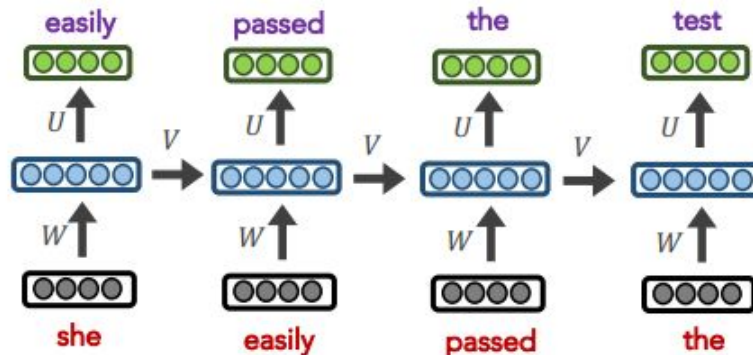
word embeddings (type-based)

approaches:

- count-based/DSMs (e.g., SVD, LSA)
- Predictive models (e.g., word2vec, GloVe)

TOKEN-BASED

contextualized embeddings are distinct for every occurrence of a word, completely dependent on its context



Bidirectional LSTM

contextualized embeddings (token-based)

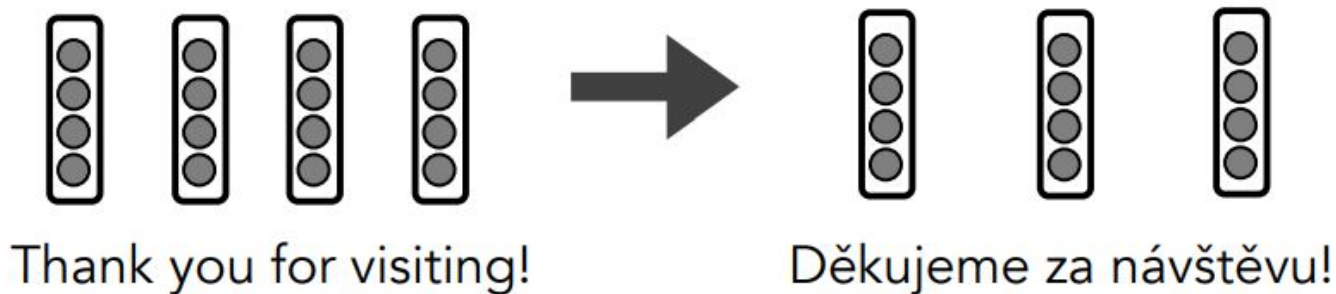
approaches:

- Predictive models (e.g., BiLSTMs, GPT-2, BERT)

LSTMs do a great job of capturing “*meaning*”, which can be used for almost every task

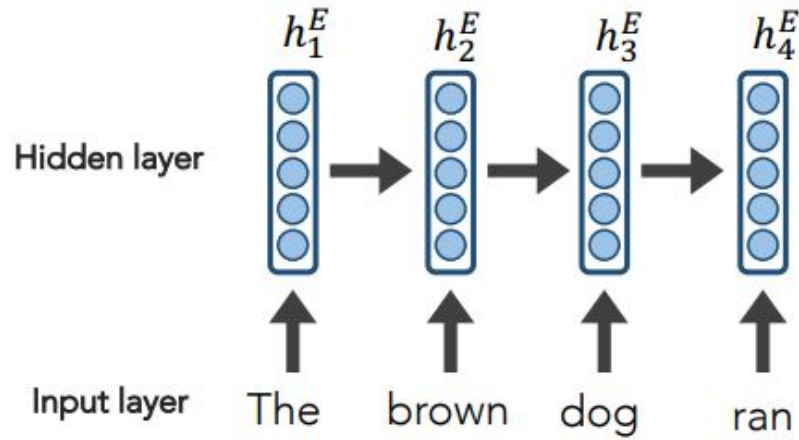
- Given a sequence of **N** words, we can produce **1** output ➡ Sentiment Analysis
- Given a sequence of **N** words, we can produce **N** outputs ➡ NER Recognition
- What if we wish to have **M** outputs? ➡ Language Translation

We want to produce a **variable-length** output
(e.g., **n** $\rightarrow$ **m** predictions)



- If our input is a sentence in **Language A**, and we wish to translate it to **Language B**, it is clearly sub-optimal to translate word by word (like our current models are suited to do).
- Instead, let a **sequence** of tokens be the unit that we ultimately wish to work with (a sequence of length **N** may emit a sequences of length **M**)
- **Seq2seq** models are comprised of **2 RNNs**: 1 encoder, 1 decoder

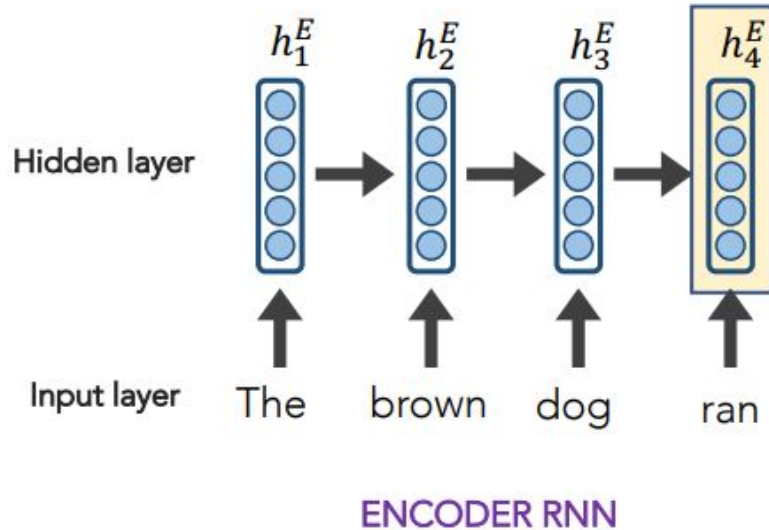
Neural Machine Translation - LSTM Based



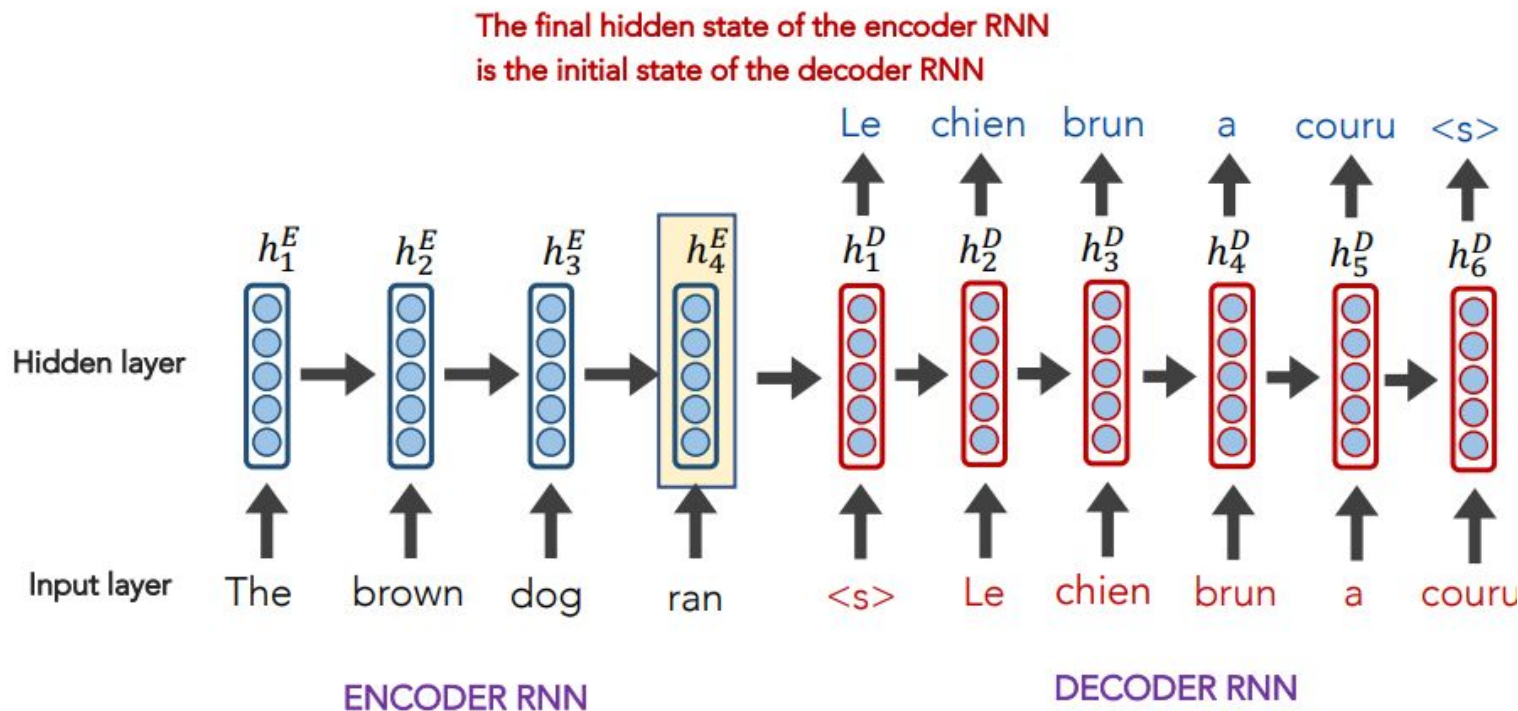
ENCODER RNN

Neural Machine Translation - LSTM Based

The final hidden state of the encoder RNN
is the initial state of the decoder RNN



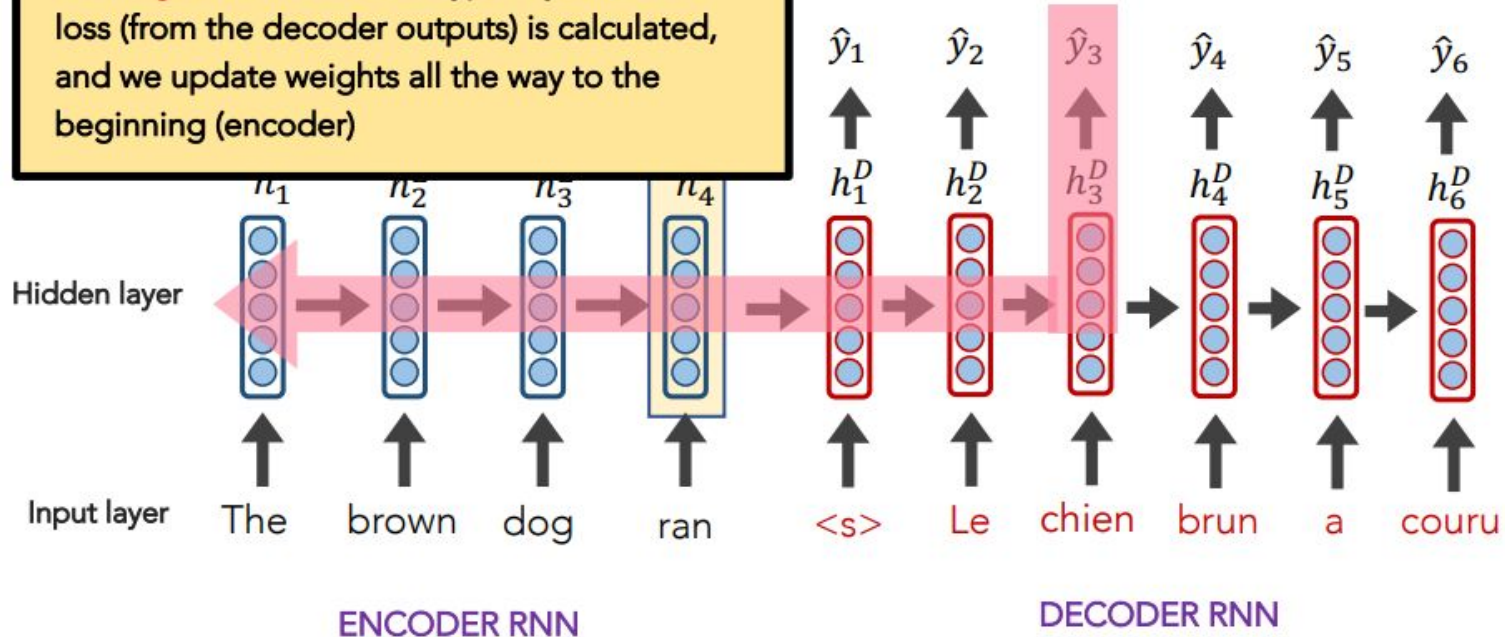
Neural Machine Translation - LSTM Based



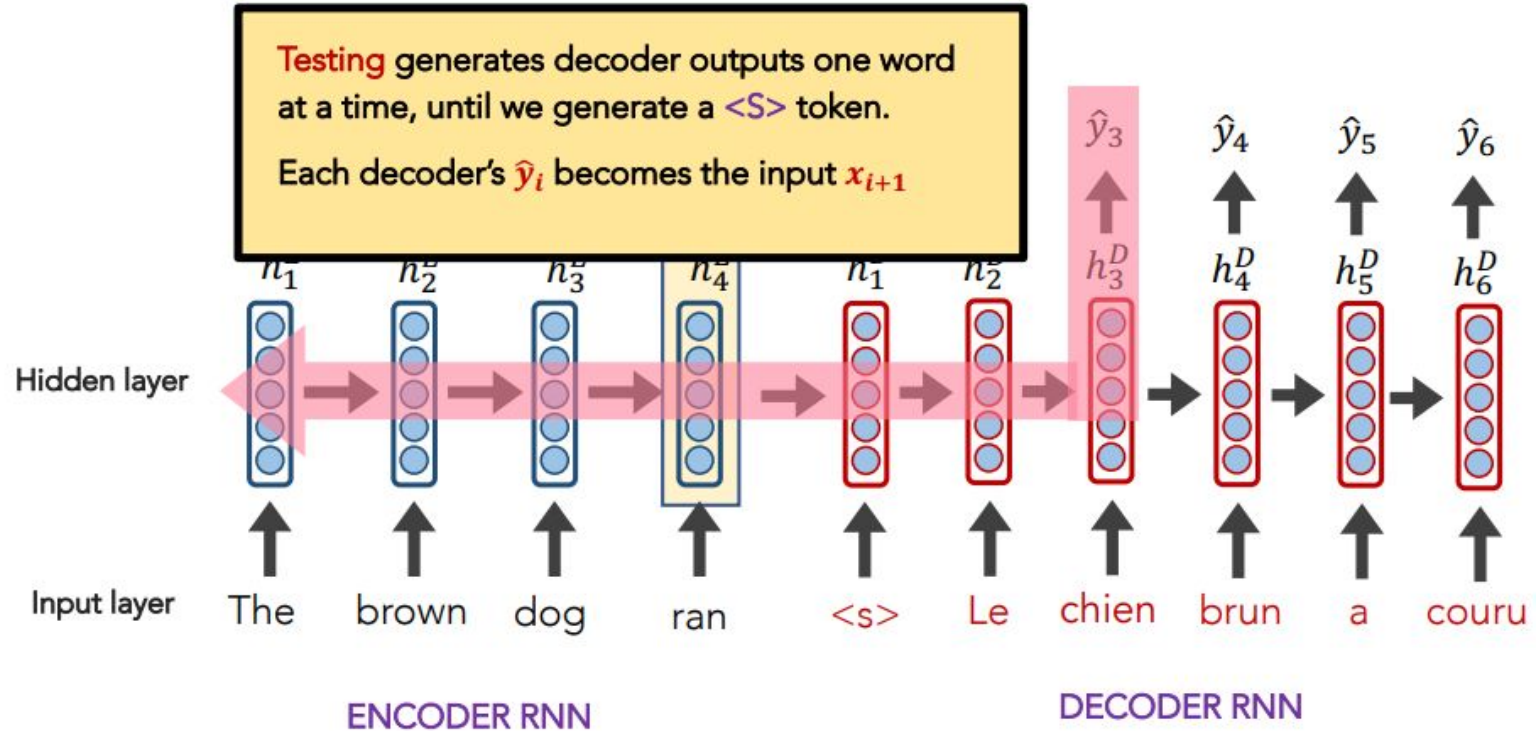
Neural Machine Translation - LSTM Based

Sequence-to-Sequence (seq2seq)

Training occurs like RNNs typically do; the loss (from the decoder outputs) is calculated, and we update weights all the way to the beginning (encoder)



Neural Machine Translation - LSTM Based

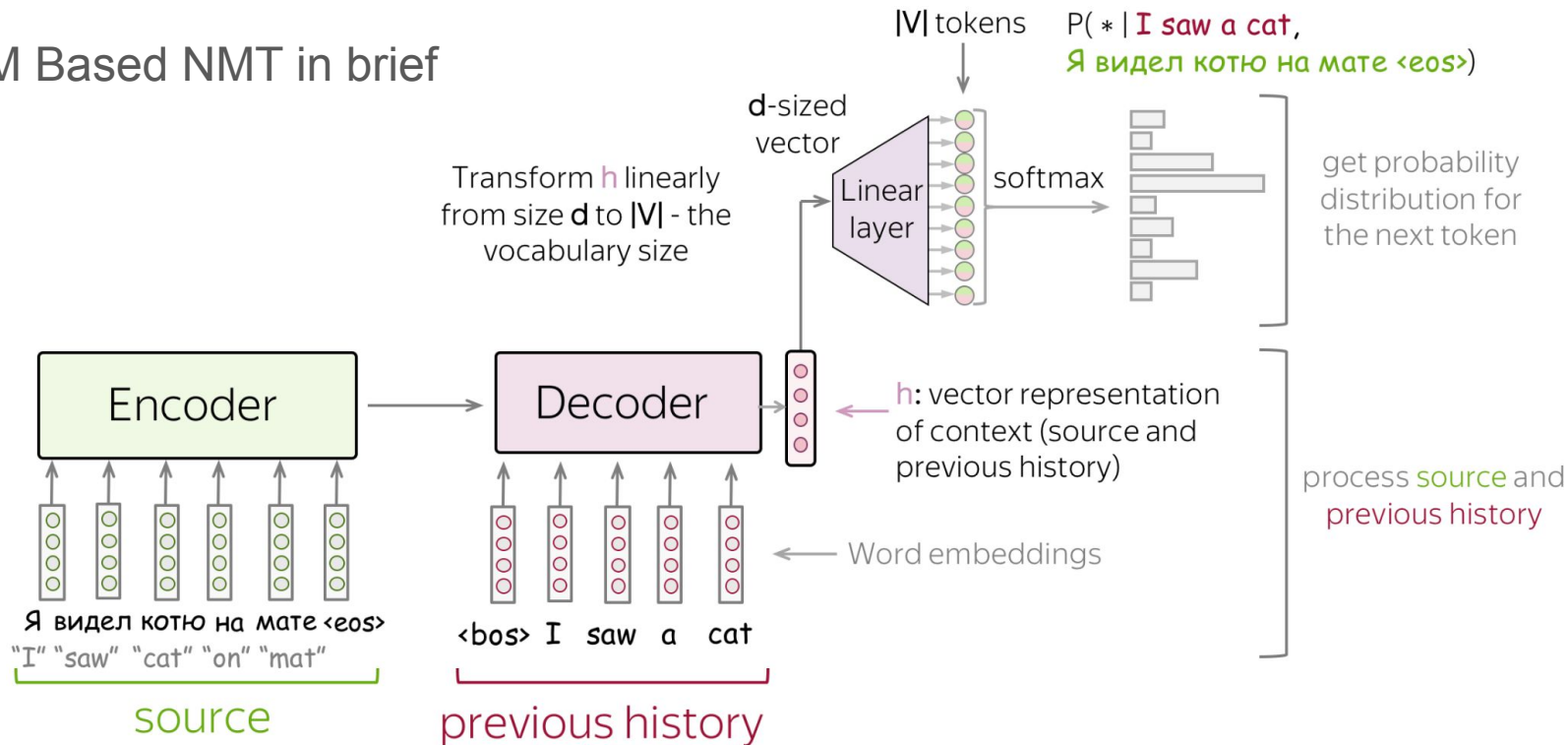


Animation! NMT

https://lena-voita.github.io/resources/lectures/seq2seq/general/enc_dec_prob_idea.mp4

Neural Machine Translation - LSTM Based

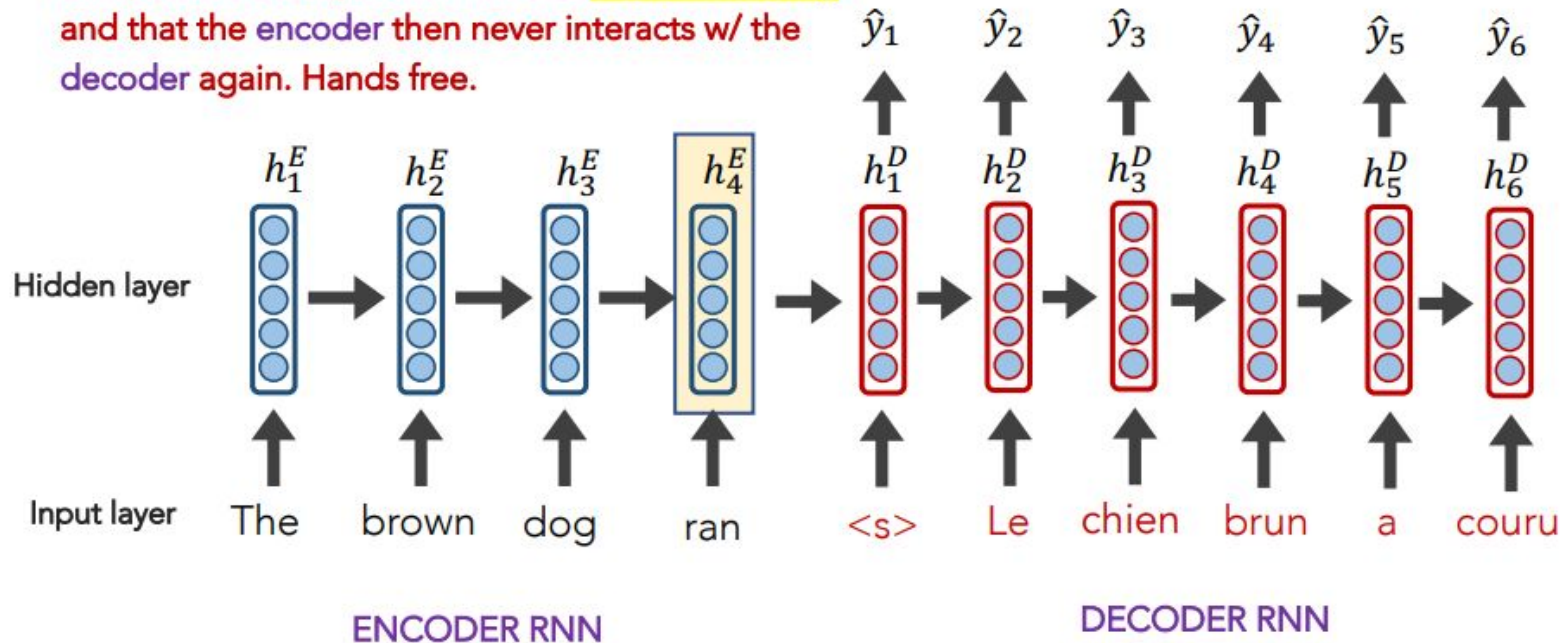
LSTM Based NMT in brief



See any issues with this traditional **seq2seq** paradigm?

Neural Machine Translation - LSTM-Attention Based ideas

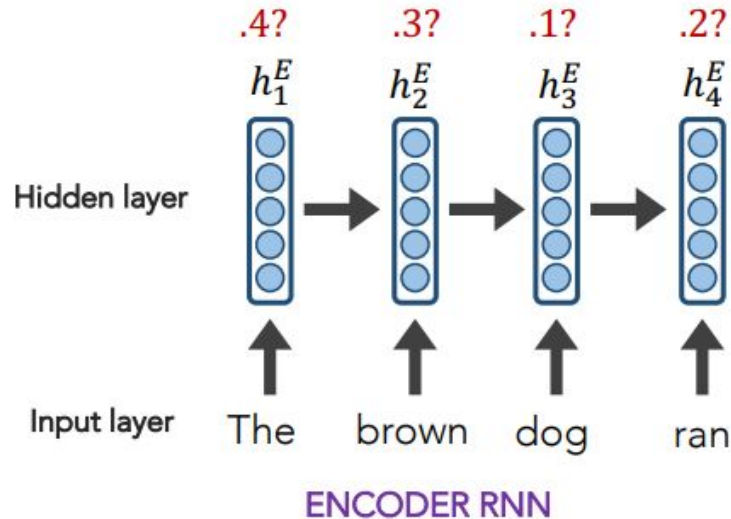
It's crazy that the entire "meaning" of the 1st sequence is expected to be packed into this **one embedding**, and that the encoder then never interacts w/ the decoder again. Hands free.



Instead, what if the decoder, at each step, pays **attention** to a *distribution* of all of the encoder's hidden states?

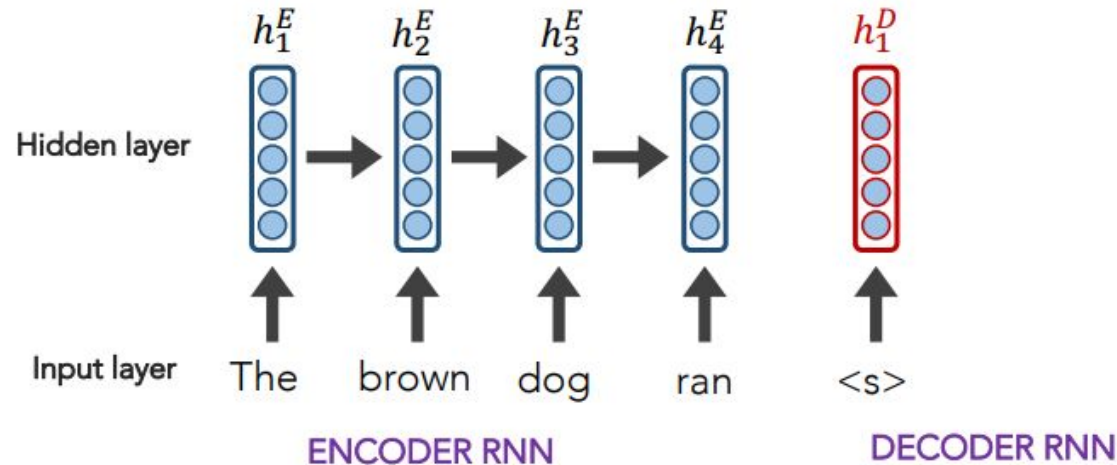
Intuition: when we (humans) translate a sentence, we don't just consume the original sentence then regurgitate in a new language; we **continuously look back at the original** while focusing on **different parts**.

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?



Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

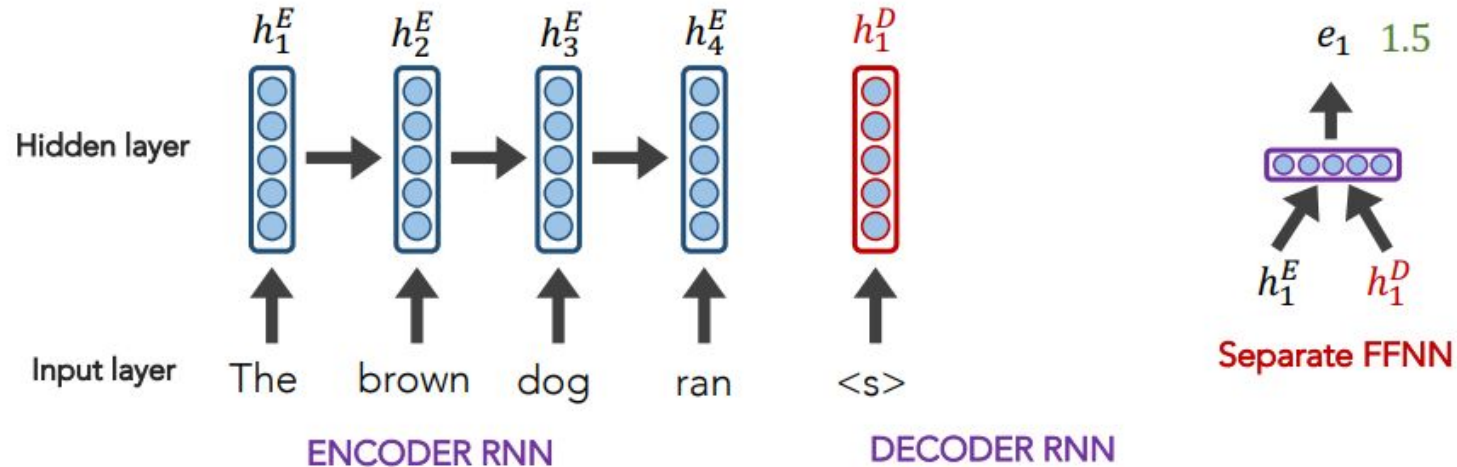
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



Neural Machine Translation - LSTM-Attention Based

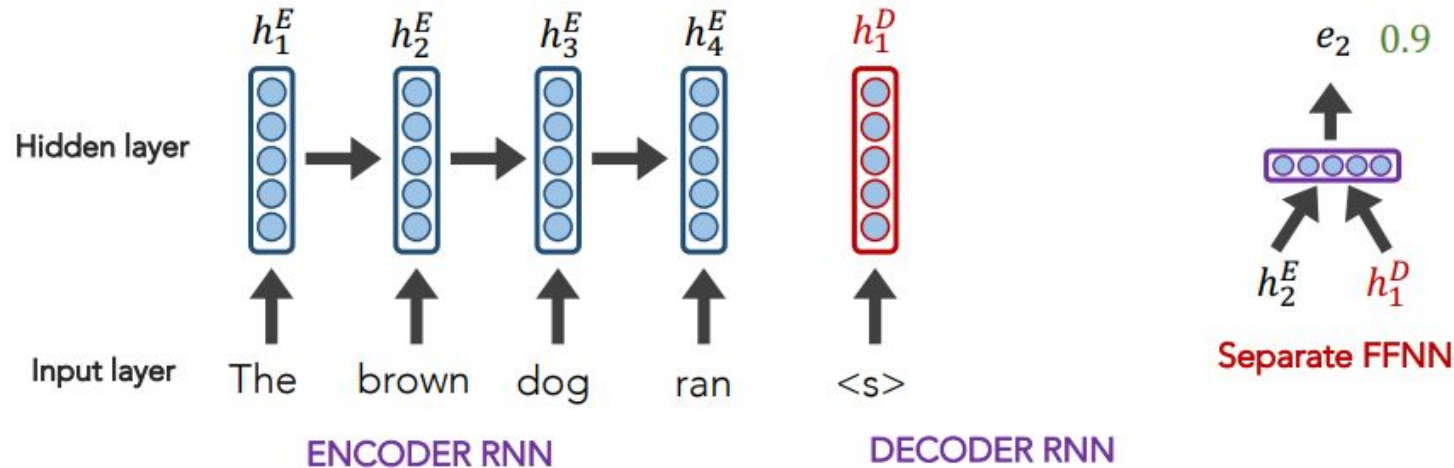
Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

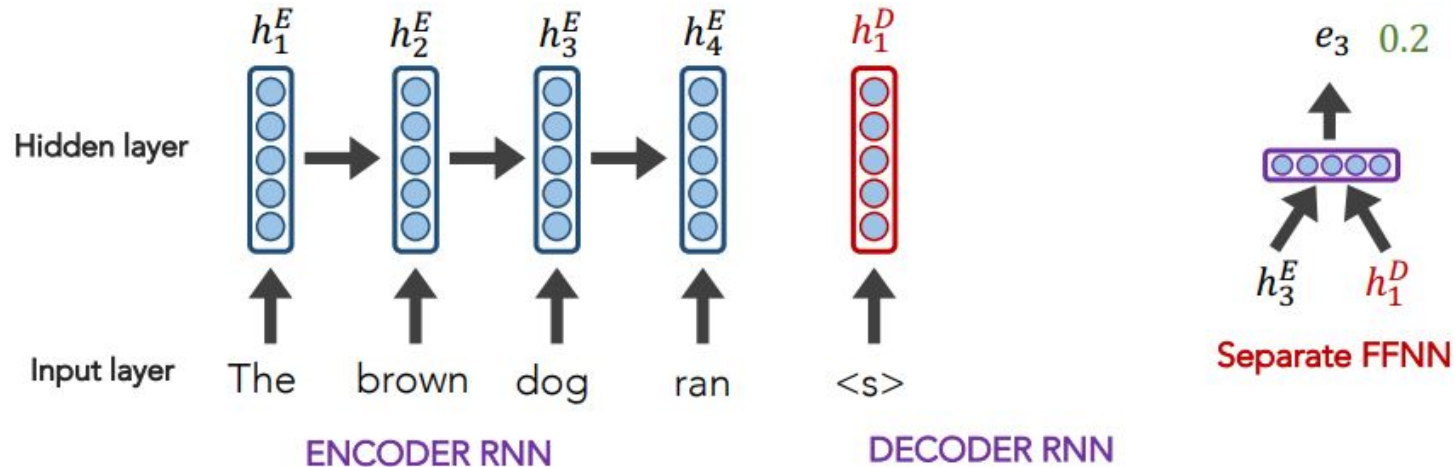
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



Neural Machine Translation - LSTM-Attention Based

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

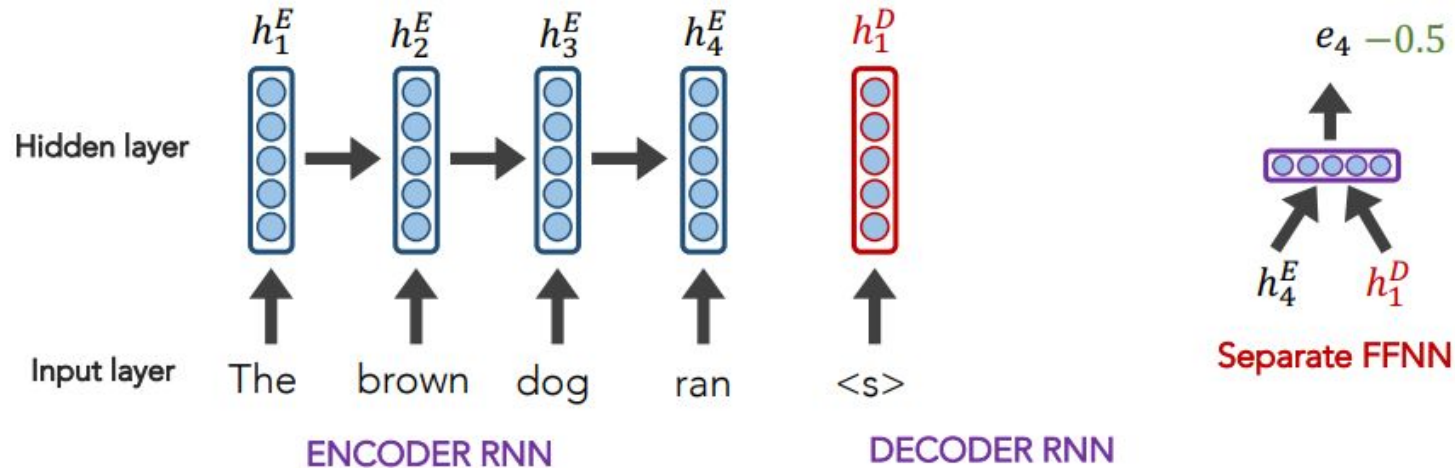
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



Neural Machine Translation - LSTM-Attention Based

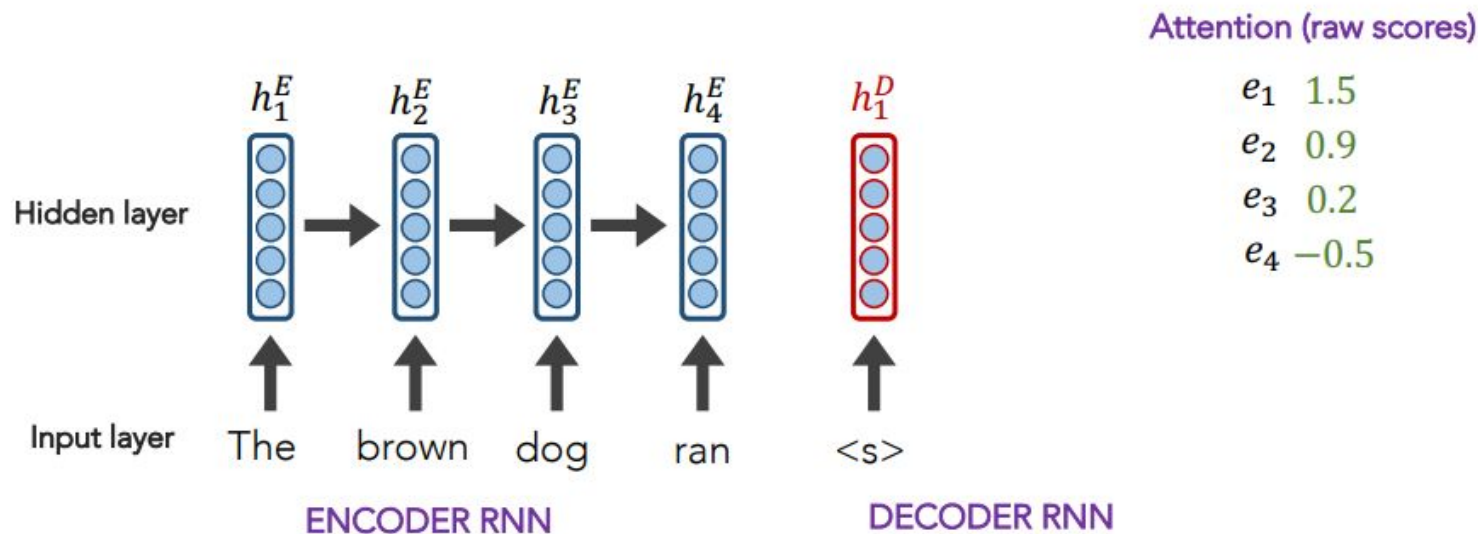
Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



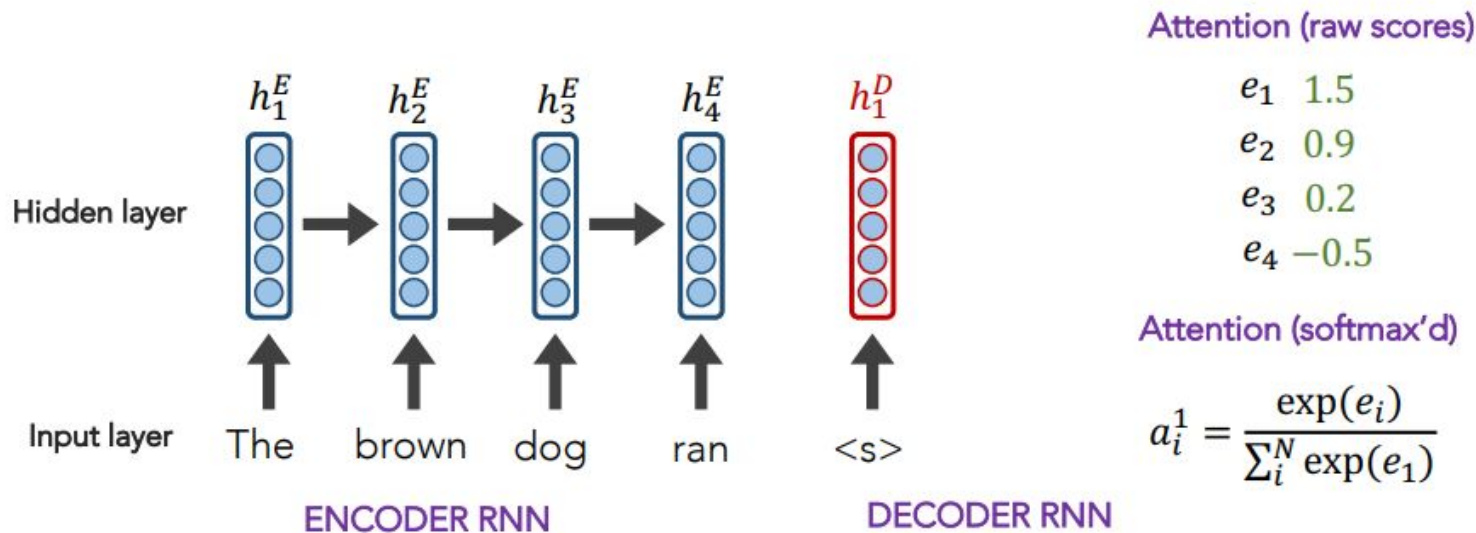
Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

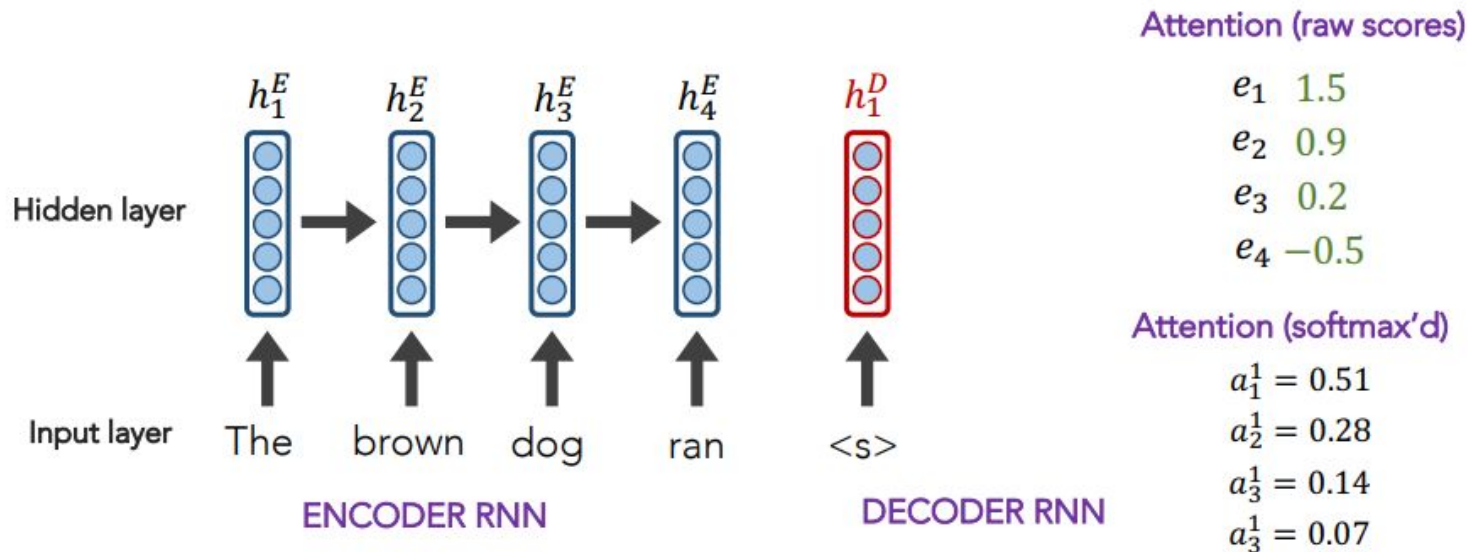
A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!



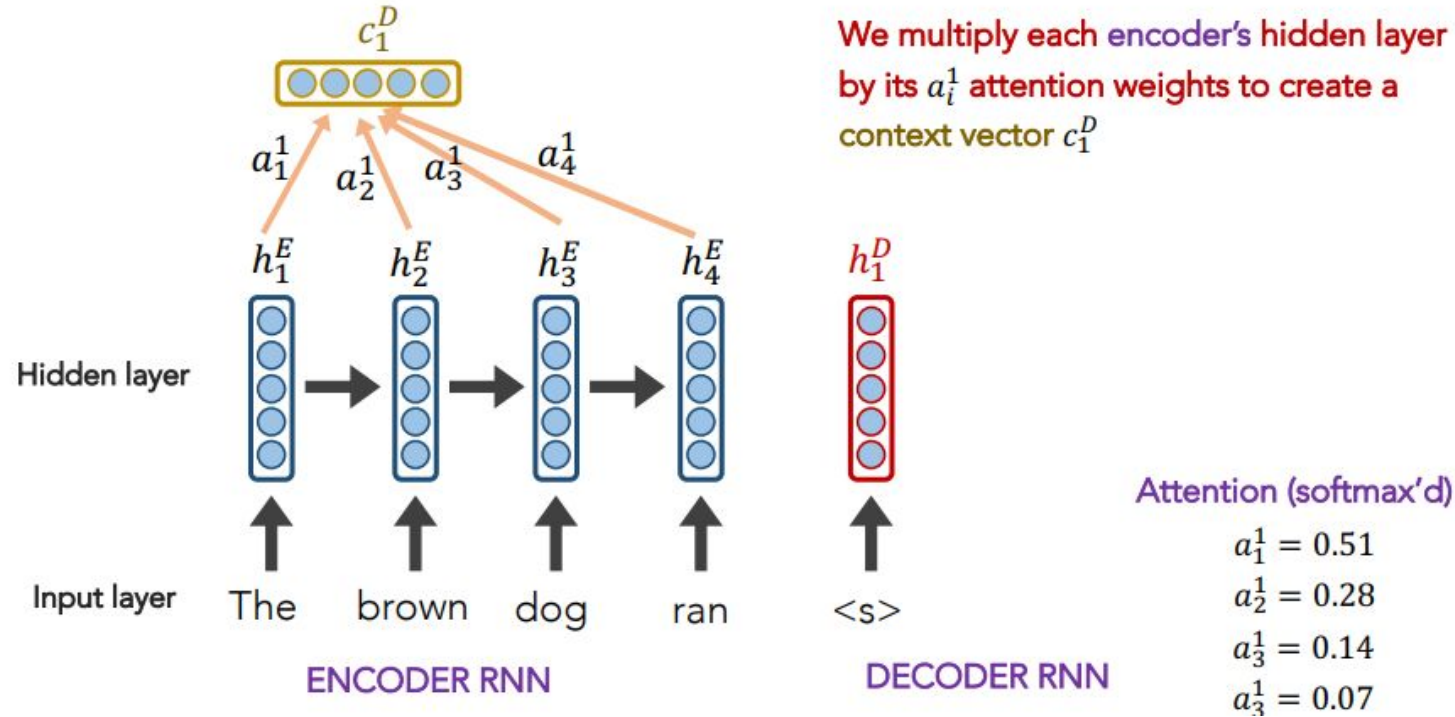
Neural Machine Translation - LSTM-Attention Based

Q: How do we determine how much to pay attention to each of the encoder's hidden layers?

A: Let's base it on our decoder's current hidden state (our current representation of meaning) and all of the encoder's hidden layers!

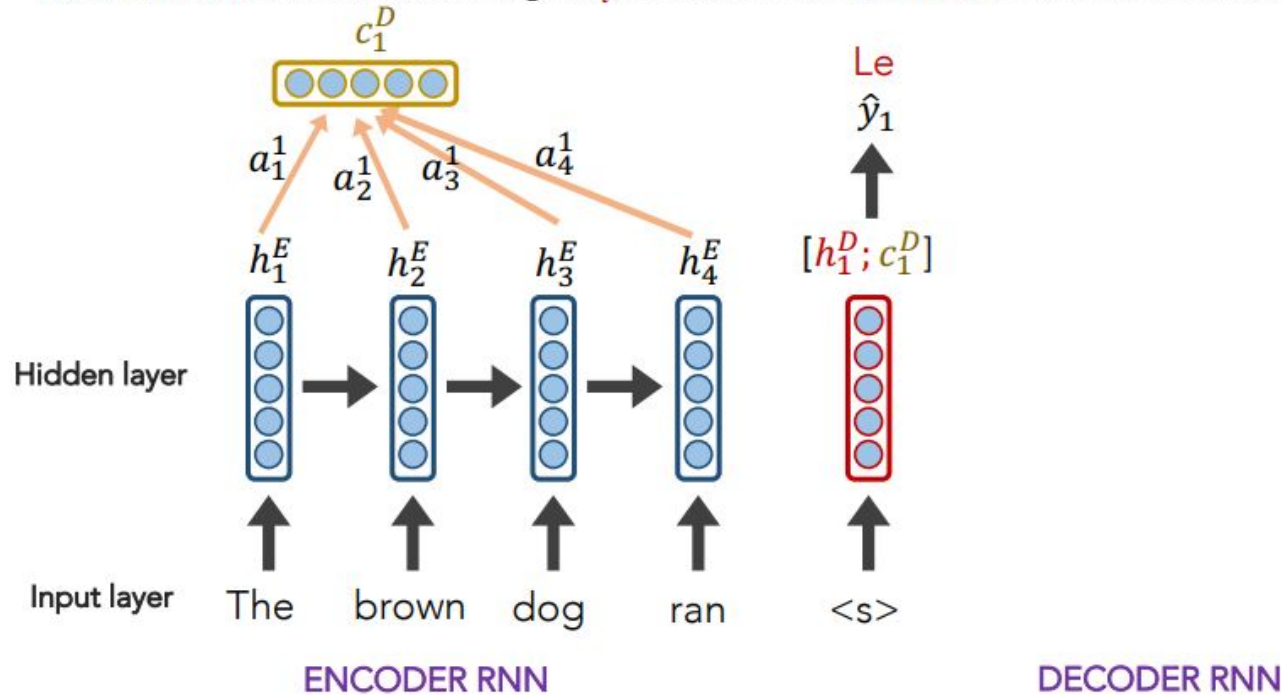


Neural Machine Translation - LSTM-Attention Based



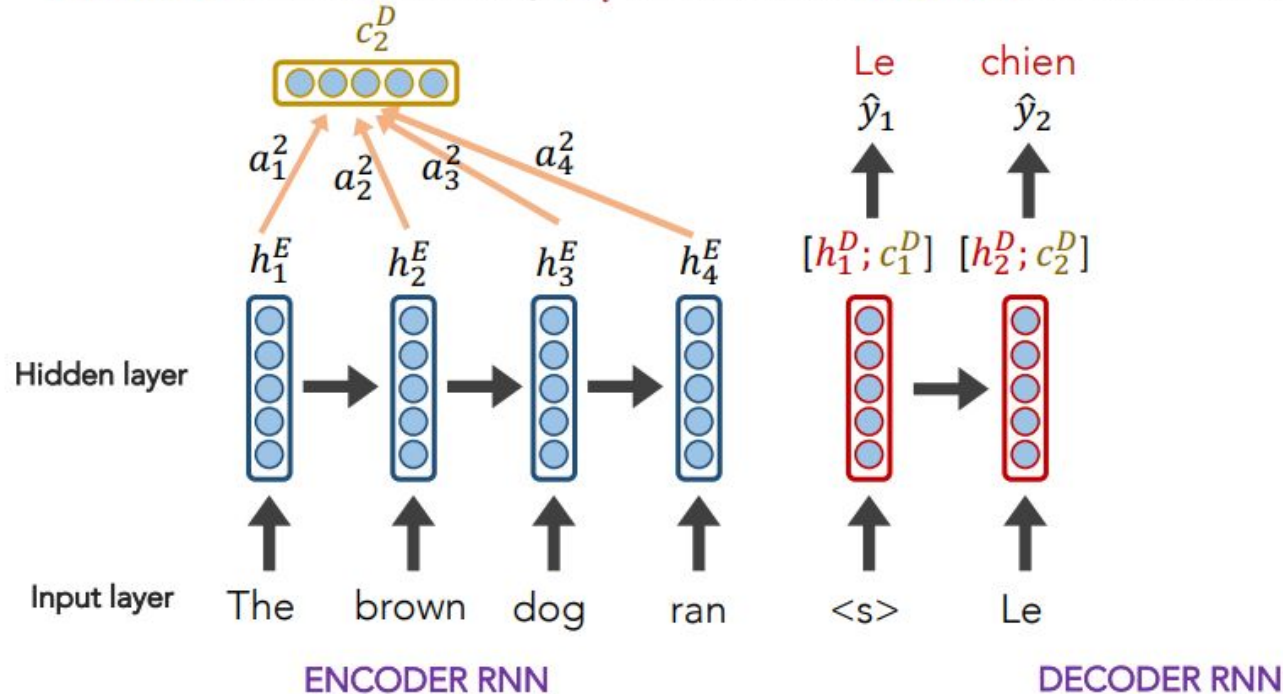
Neural Machine Translation - LSTM-Attention Based

REMEMBER: each attention weight a_i^j is based on the **decoder's** current hidden state, too.



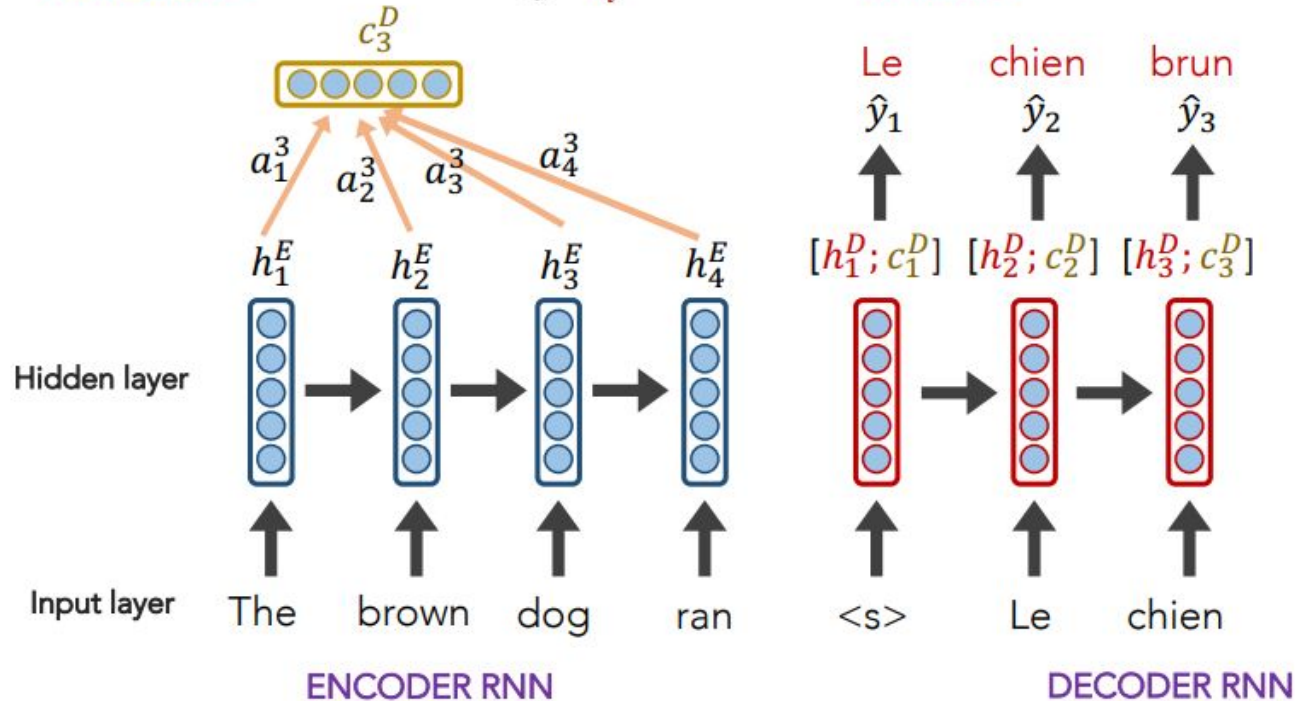
Neural Machine Translation - LSTM-Attention Based

REMEMBER: each attention weight a_i^j is based on the **decoder's** current hidden state, too.



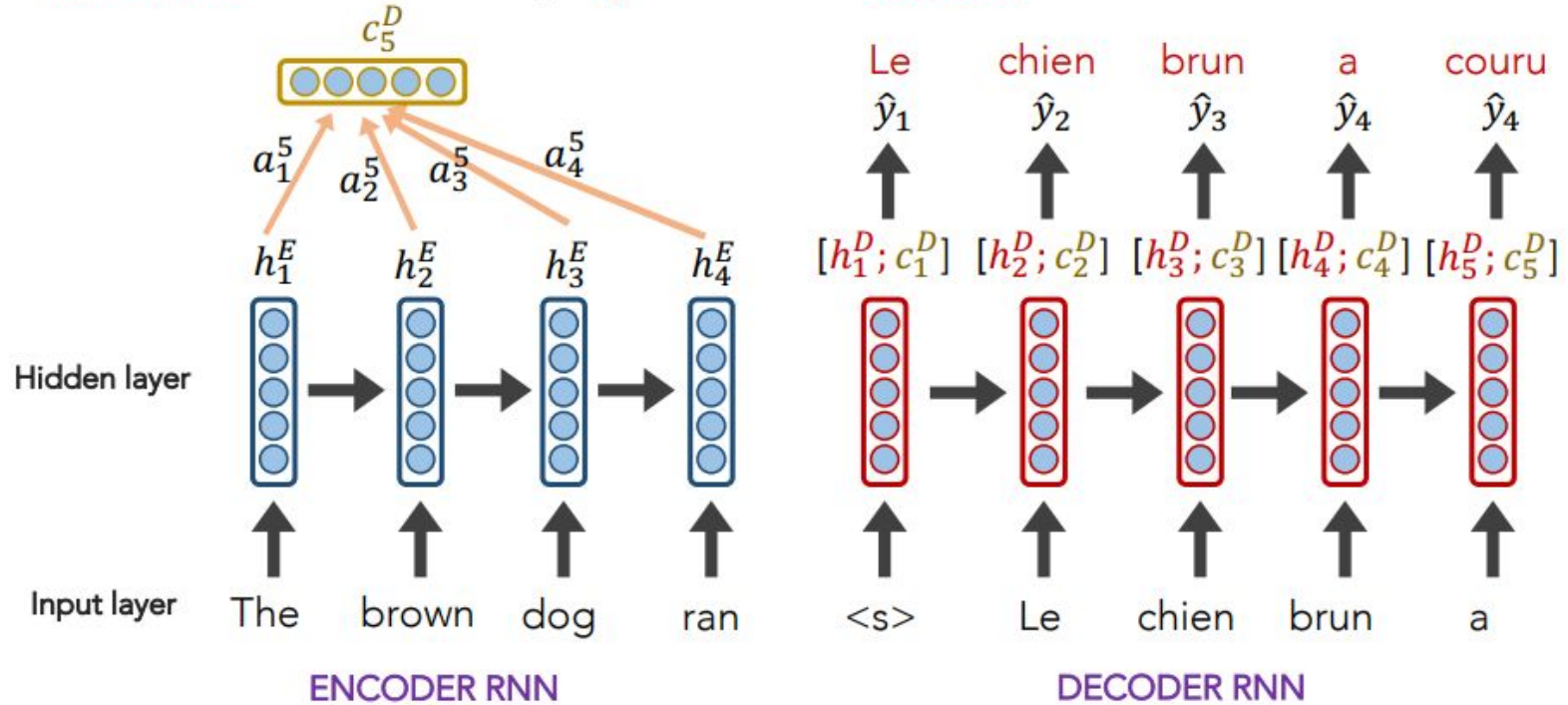
Neural Machine Translation - LSTM-Attention Based

REMEMBER: each attention weight a_i^j is based on the **decoder's** current hidden state, too.



Neural Machine Translation - LSTM-Attention Based

REMEMBER: each attention weight a_i^j is based on the **decoder's** current hidden state, too.



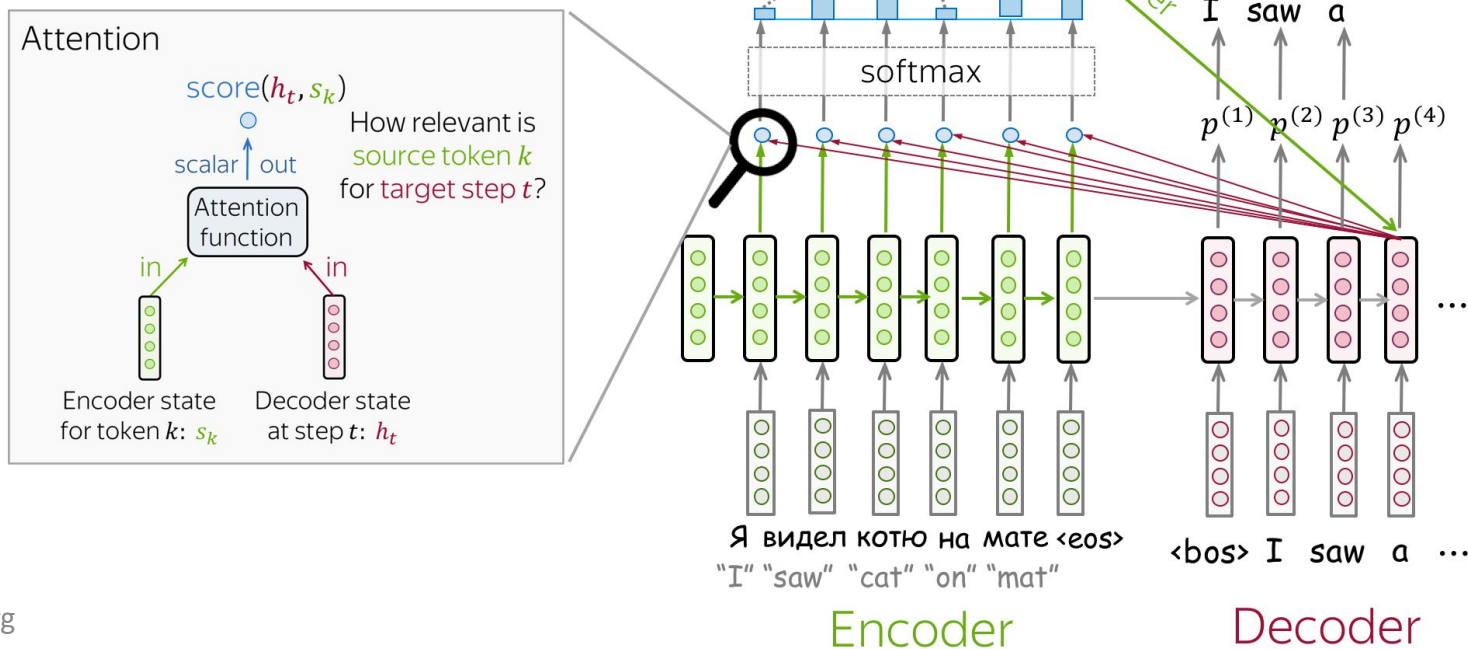
Neural Machine Translation - LSTM-Attention Based

LSTM-Attention Based NMT In Brief

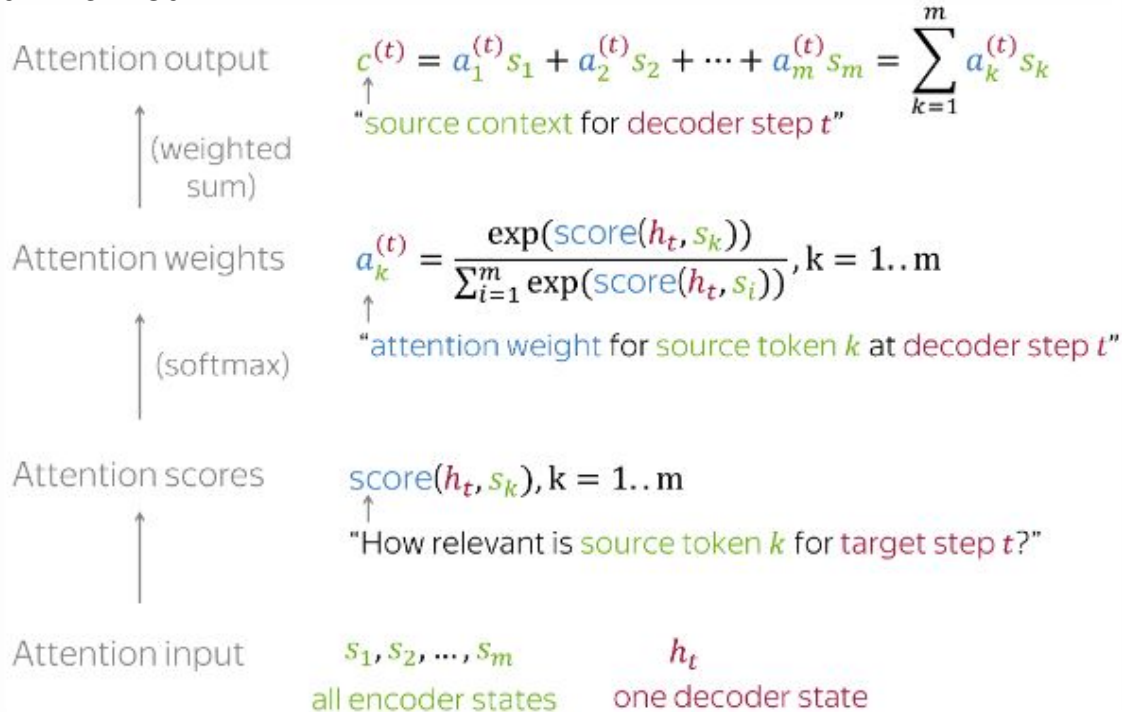
Attention output: weighted sum of encoder states with attention weights

Attention weights: distribution over source tokens

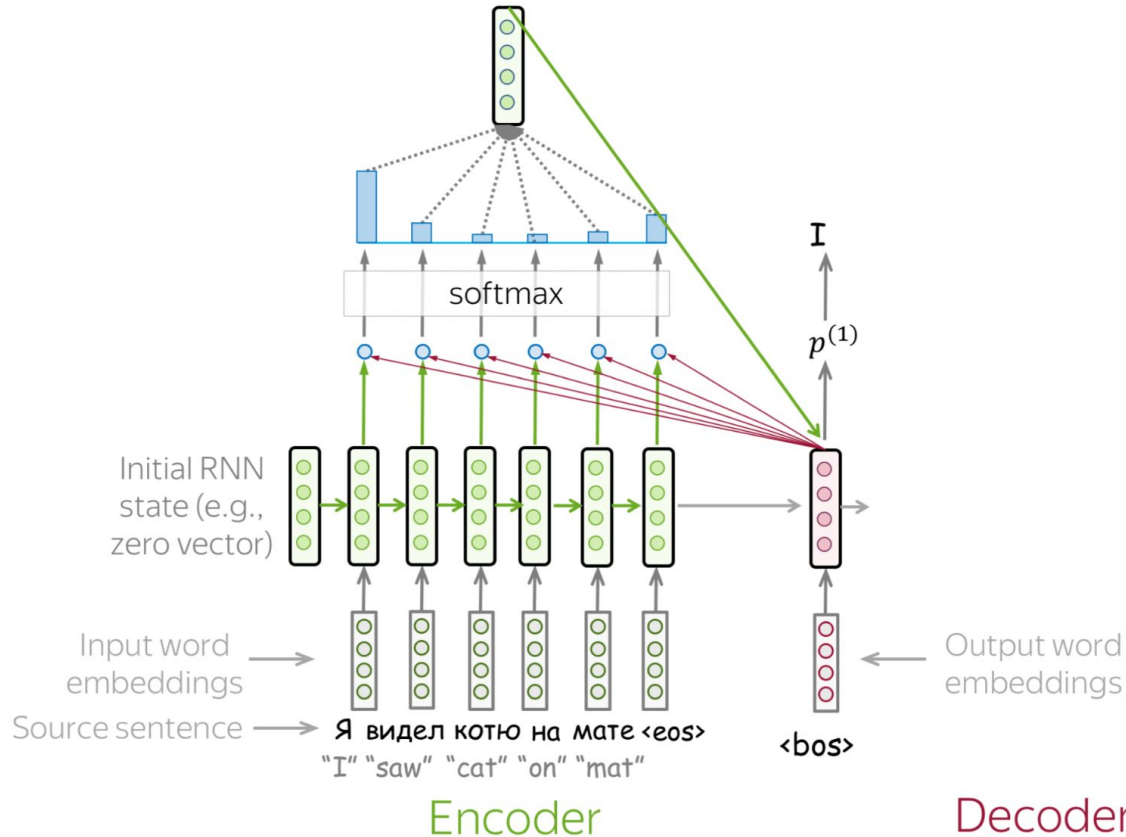
A model can learn to “pay attention” to the most relevant source tokens for each step



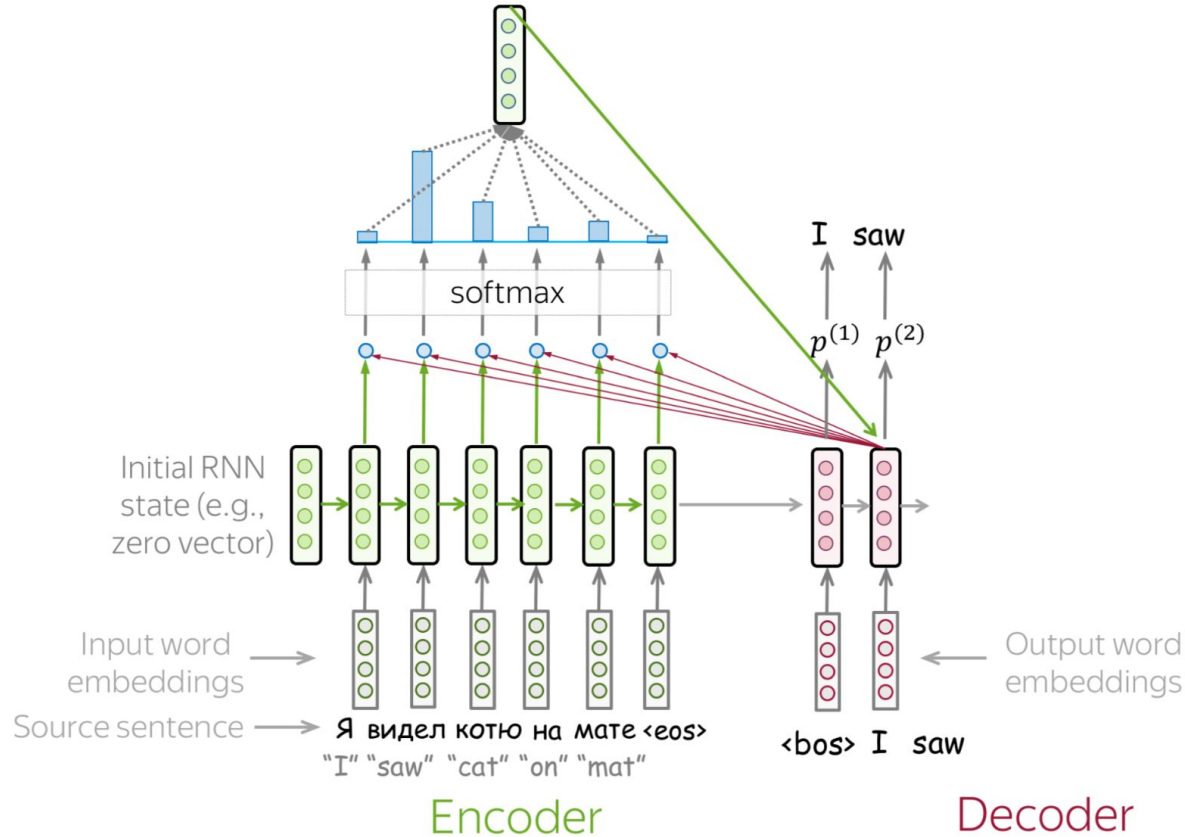
Attention calculation summarized



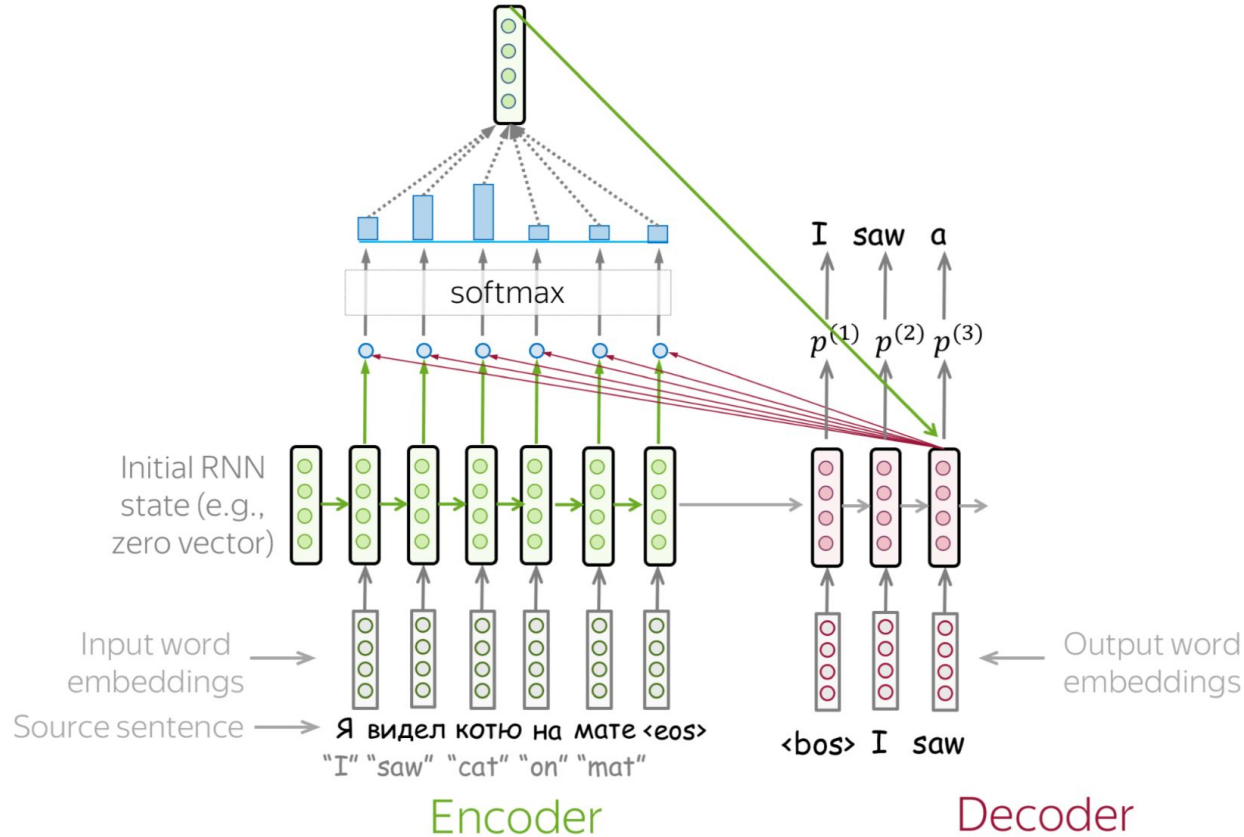
Neural Machine Translation - LSTM-Attention Based



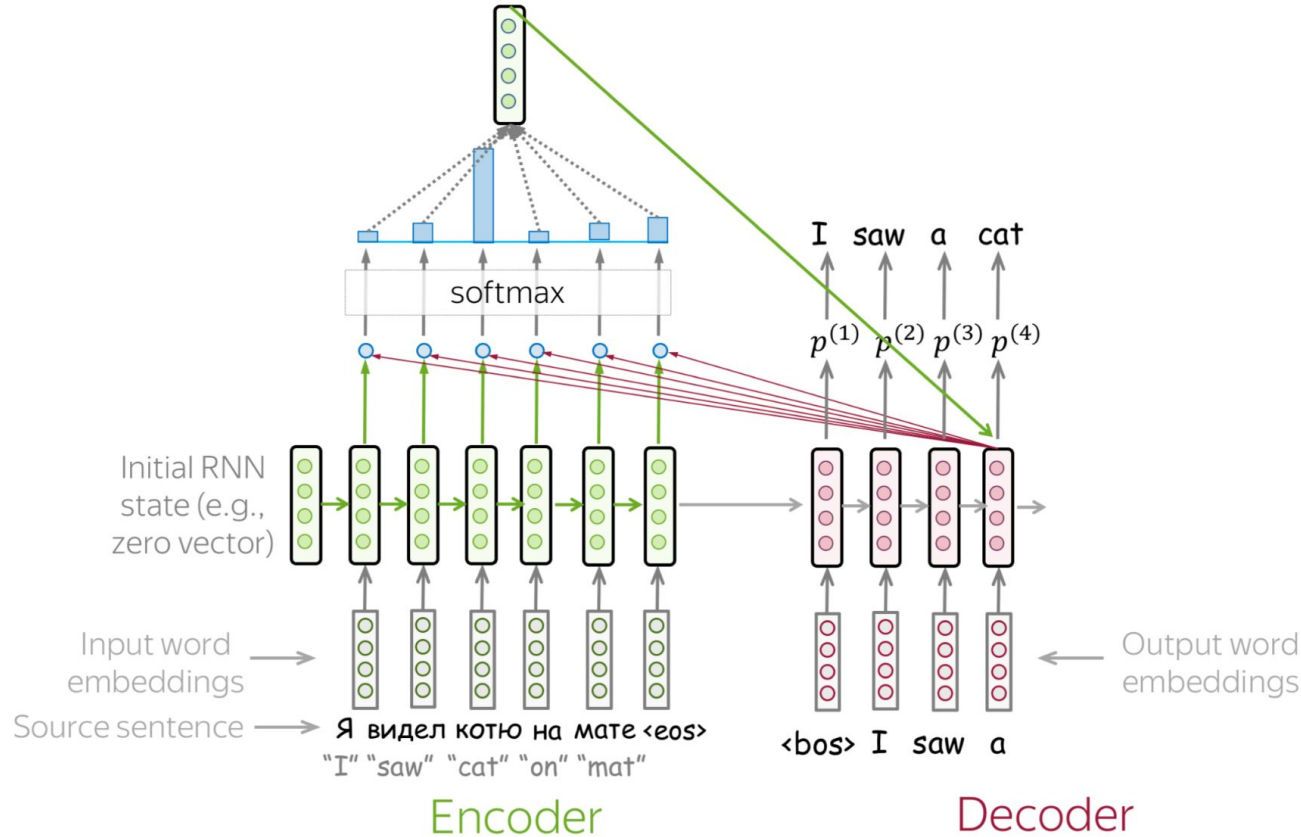
Neural Machine Translation - LSTM-Attention Based



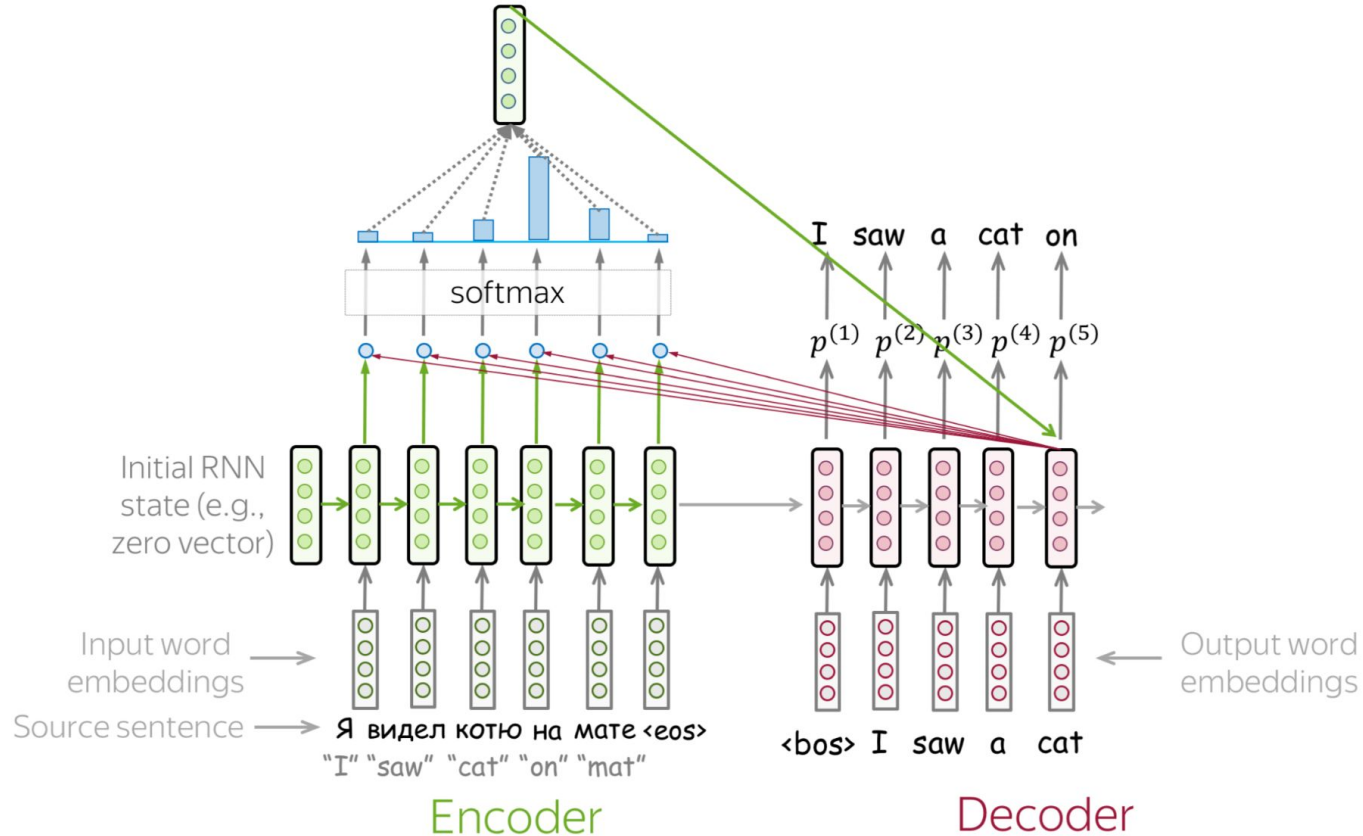
Neural Machine Translation - LSTM-Attention Based



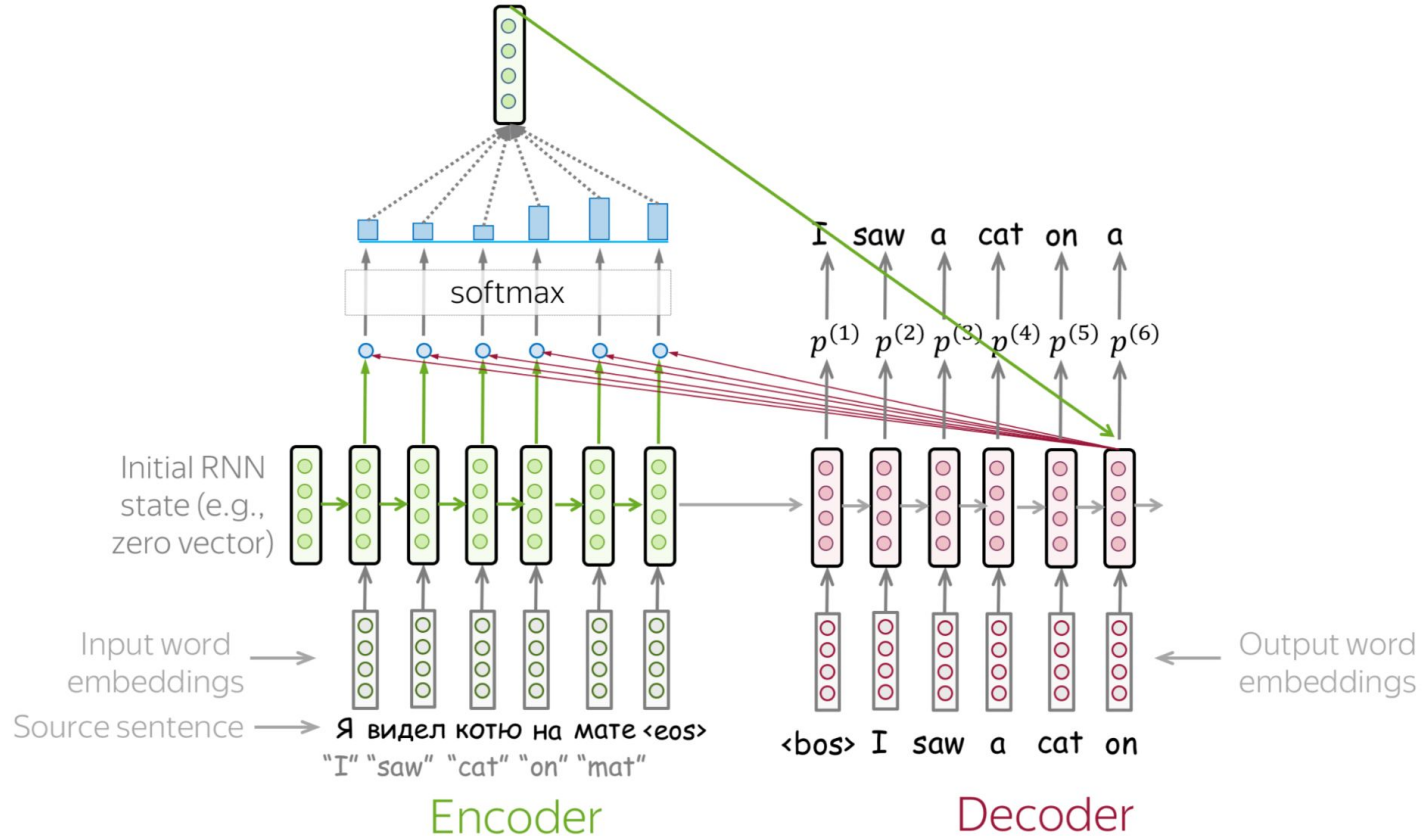
Neural Machine Translation - LSTM-Attention Based



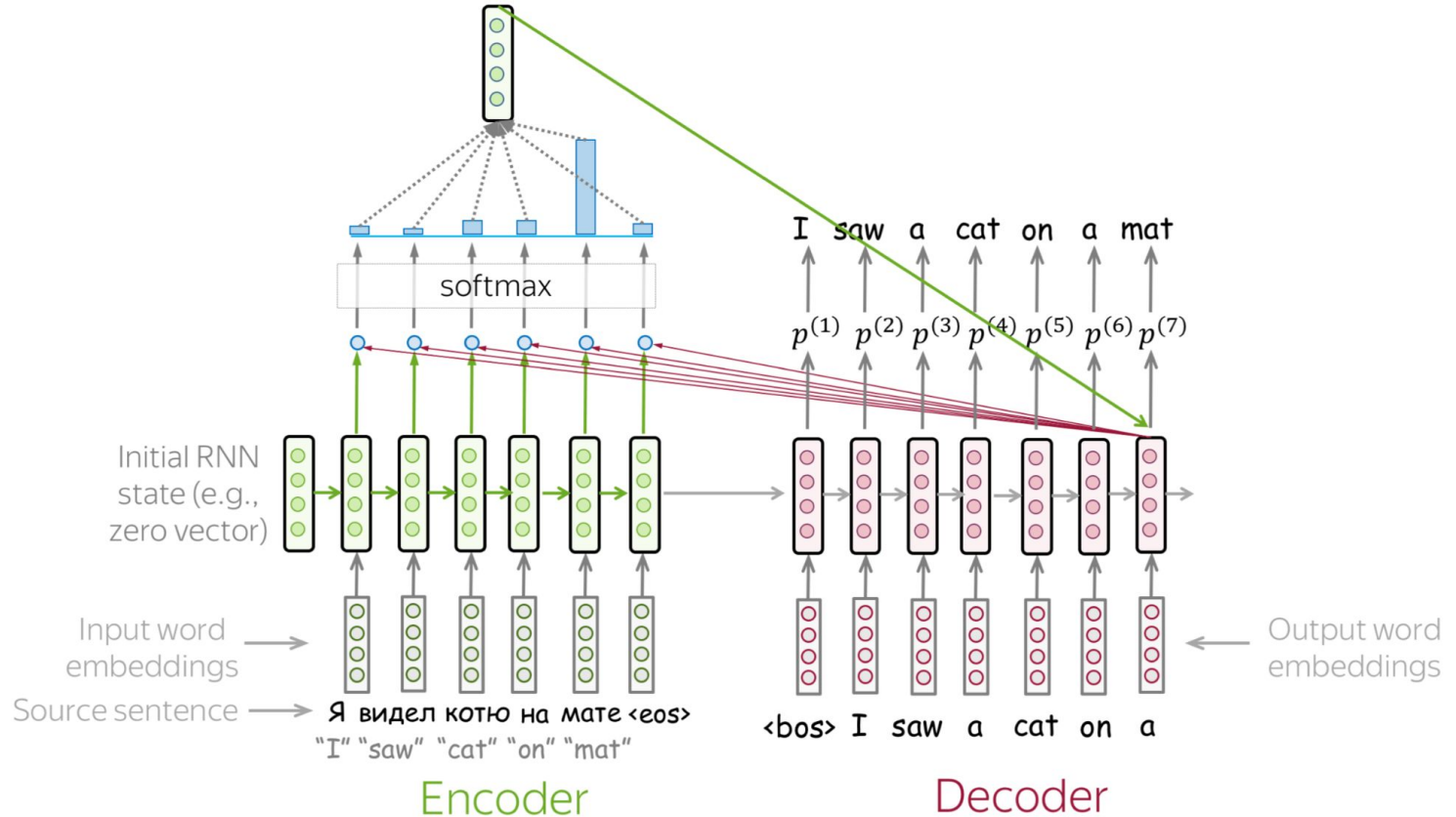
Neural Machine Translation - LSTM-Attention Based



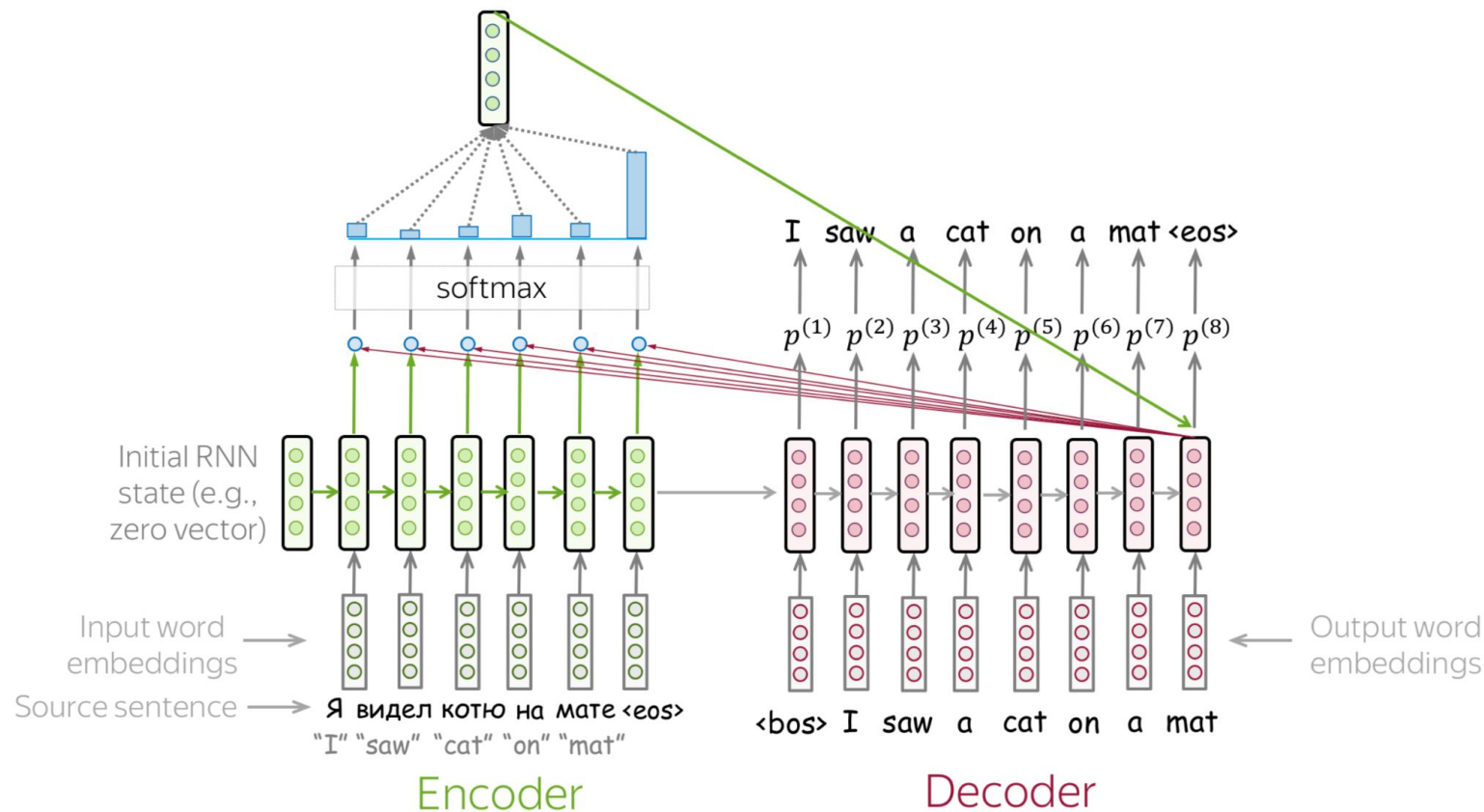
Neural Machine Translation - LSTM-Attention Based

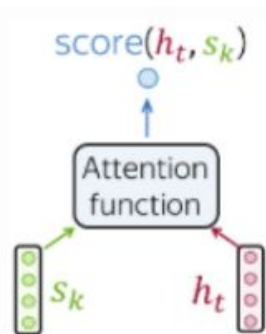


Neural Machine Translation - LSTM-Attention Based



Neural Machine Translation - LSTM-Attention Based





Popular Attention Scoring functions:

Dot-product

$$h_t^T \times s_k$$

$$\text{score}(h_t, s_k) = h_t^T s_k$$

Bilinear

$$h_t^T \times [W] \times s_k$$

$$\text{score}(h_t, s_k) = h_t^T W s_k$$

Multi-Layer Perceptron

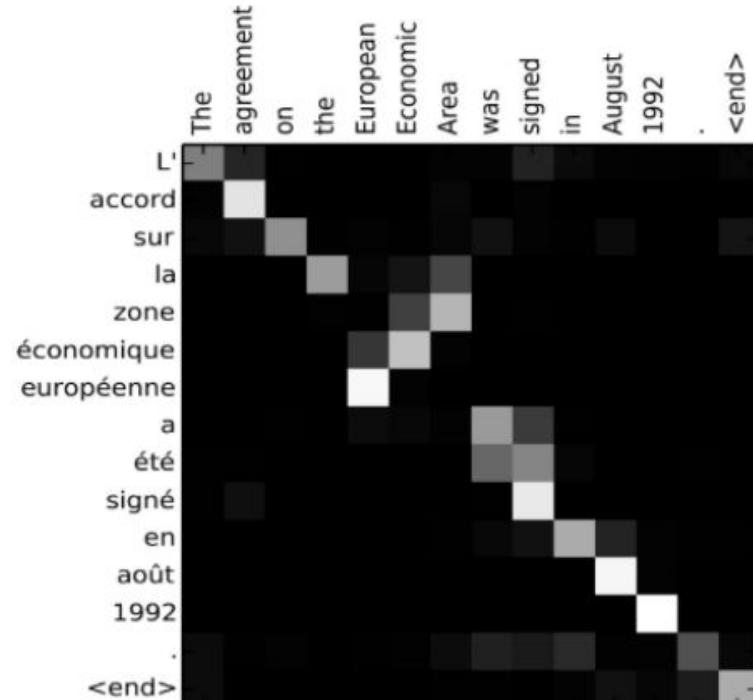
$$w_2^T \times \tanh \left[W_1 \times \begin{bmatrix} h_t \\ s_k \end{bmatrix} \right]$$

$$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$$

Neural Machine Translation - LSTM-Attention Based

Attention:

- greatly improves seq2seq results
- allows us to visualize the contribution each **encoding** word gave for each **decoder's** word



Summaries

- **LSTMs** yielded state-of-the-art results on most NLP tasks (2014-2018)
- **seq2seq+Attention** was an even more revolutionary idea (Google Translate used it)
- **Attention** allows us to place appropriate weight to the encoder's hidden states
- But, **LSTMs** require us to iteratively scan each word and wait until we're at the end before we can do anything